



Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Ingeniería Informática

Simulador web de escenarios de movilidad urbana mediante técnicas de Inteligencia Artificial

Iván Vicente Hernández García de Mora

Noviembre, 2025

TRABAJO FIN DE MÁSTER

Máster Universitario en Ingeniería Informática

Simulador web de escenarios de movilidad urbana mediante técnicas de Inteligencia Artificial

Autor: Iván Vicente Hernández García de Mora

Director: Nombre del director

Codirector: Nombre del codirector

*Aquí va la dedicatoria que cada cual
quiera escribir. El ancho se controla
manualmente*

Declaración de autoría

Yo, ... , con DNI ... , declaro que soy el único autor del trabajo fin de grado titulado “ ... ”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a ... de ... de 20 ...

Fdo.: Iván Vicente Hernández García de Mora

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Índice general

Índice de figuras

Índice de tablas

Índice de algoritmos

Índice de listados de código

1. Introducción

1.1. Contexto y motivación

1.2. Objetivos del TFM

1.2.1. Objetivo general

Desarrollar un prototipo de simulador web de movilidad urbana que integre modelos de predicción basados en Machine Learning y servicios de enrutado, con el fin de analizar el impacto de distintas políticas de transporte en el reparto modal de los viajes y generar métricas de apoyo para la planificación y la toma de decisiones.

1.2.2. Objetivos específicos

- Entrenar y validar modelos de Machine Learning con datos de movilidad para predecir la elección modal de los usuarios en función de variables sociodemográficas, de tiempo y de coste.
- Integrar servicios de enrutado (OSRM y OpenTripPlanner) para obtener tiempos y costes realistas entre pares origen-destino en el entorno urbano.
- Desarrollar una aplicación web interactiva que permita definir viajes sobre un mapa, simular el comportamiento de los usuarios y visualizar resultados en un cuadro de mando.
- Implementar un módulo de análisis de escenarios *what-if* para modificar parámetros de política (coste del coche, tarifas del transporte público, frecuencia de autobuses, etc.) y observar su efecto en el reparto modal.

-
- Generar métricas de impacto (reparto modal, tiempo medio de viaje y estimación de emisiones de CO₂) como apoyo a la toma de decisiones en movilidad urbana.

1.2.3. Competencias trabajadas

El desarrollo del TFM contribuye a las siguientes competencias del Máster:

- **CP01:** Integrar tecnologías, aplicaciones, servicios y sistemas de la Ingeniería Informática en un contexto multidisciplinar.
- **CP05:** Aplicar métodos matemáticos, estadísticos y de inteligencia artificial para modelar y desarrollar sistemas inteligentes.
- **CP07:** Realizar un proyecto integral original de carácter profesional que sintetice las competencias adquiridas en el plan de estudios.

1.3. Alcance y limitaciones

2. Estado del arte y marco tecnológico

2.1. Fundamentos y enfoques

La movilidad urbana se ha consolidado como un ámbito de estudio en el que confluyen la ingeniería del transporte, la analítica de datos y la inteligencia artificial. En este contexto, los simuladores actuales no se limitan a representar mapas o a calcular rutas aisladas, sino que actúan como herramientas para evaluar políticas y escenarios de movilidad, permitiendo analizar de forma anticipada cómo cambios en los costes, en la oferta de transporte o en las condiciones operativas pueden modificar el comportamiento de la población.

Este tipo de sistemas exige combinar varias capas de conocimiento. Por un lado, la capa de datos, que determina la calidad y la trazabilidad de la información sobre la red de transporte y los servicios de transporte público asociados. Por otro, la capa de enrutado, que permite transformar un par origen-destino en variables de servicio observables (tiempo, distancia, segmentos de viaje, transbordos o coste aproximado). Finalmente, la capa de modelado, en la que dichas variables se integran con atributos sociodemográficos para estimar la elección modal y construir escenarios *what-if*. Esta arquitectura en capas es coherente con la evolución reciente de herramientas de simulación en investigación y planificación [Horni et al., 2016, Lopez et al., 2018].

2.2. Datos abiertos para movilidad

El desarrollo de los sistemas descritos en el apartado anterior requiere la disponibilidad de grandes volúmenes de datos fiables y reutilizables. Por este motivo, en los últimos años han adquirido gran protagonismo los conjuntos de datos abiertos en movilidad, ya que

permiten construir, validar y reproducir modelos de análisis sin depender de fuentes propietarias cerradas. En este contexto, OpenStreetMap y GTFS se han consolidado como los dos pilares más utilizados para representar, respectivamente, la red urbana y la oferta de transporte público.

2.2.1. OpenStreetMap

OpenStreetMap (OSM) es una base de datos geográfica abierta y colaborativa ampliamente utilizada en proyectos de movilidad, tanto en investigación como en aplicaciones operativas. La propia fundación del proyecto define OSM como una fuente de datos abiertos reutilizable por terceros, con cobertura global y actualización continua por parte de la comunidad [?]. Esta naturaleza abierta resulta especialmente relevante en un trabajo experimental, ya que permite reproducir resultados a partir de un conjunto de datos público, versionable e independiente de condiciones comerciales impuestas por terceros.

En términos de calidad cartográfica, diversos estudios han mostrado que OSM puede ofrecer niveles de precisión adecuados para el análisis del transporte, aunque con cierta heterogeneidad espacial entre zonas y entre tipos de elemento, por ejemplo, entre la red viaria principal y los detalles de la infraestructura peatonal [Haklay, 2010]. Por ello, en este trabajo OSM se considera una fuente adecuada para construir la red de enrutado, teniendo en cuenta las características específicas del área de estudio al interpretar los resultados.

2.2.2. GTFS

GTFS (*General Transit Feed Specification*) es el estándar abierto dominante para publicar oferta de transporte público programada. La documentación oficial describe de forma precisa el contrato de datos entre productor y consumidor: estructura de ficheros, relaciones entre entidades y restricciones semánticas necesarias para que un planificador pueda reconstruir correctamente servicios, horarios y calendarios [MobilityData, 2026b]. En otras palabras, GTFS no es solo un formato de intercambio, sino también un marco común de interoperabilidad entre operadores, agregadores y motores de planificación.

La estructura del estándar se basa en un conjunto de ficheros fundamentales, entre los que destacan `stops.txt`, `routes.txt`, `trips.txt`, `stop_times.txt`, `calendar.txt` y `calendar_dates.txt`. Estos archivos permiten representar simultáneamente la topología de la red y la dimensión temporal del servicio [MobilityData, 2026b]. Esta separación resulta esencial, ya que el componente espacial define dónde se presta el servicio y el componente temporal determina cuándo se presta. En un entorno de simulación multimodal, ambos

planos deben mantenerse coherentes para evitar itinerarios inviables o resultados inconsistentes.

En el contexto español, el canal institucional más relevante para localizar y descargar feeds GTFS es el *Punto de Acceso Nacional de Transporte Multimodal* (NAP), gestionado por el Ministerio de Transportes y Movilidad Sostenible. La propia descripción oficial del portal lo define como el punto único nacional para concentrar información de oferta de transporte de viajeros, en línea con el marco europeo de datos de movilidad [NAP Transporte Multimodal, 2026a, NAP Transporte Multimodal, 2026b].

En este TFM, la descarga de datos para transporte urbano se realizó precisamente desde NAP. Como ejemplo de referencia nacional se consultó el conjunto de EMT Valencia y, para el caso de estudio del proyecto, el conjunto de autobuses urbanos de Toledo [EMT Valencia, 2026, ?]. En ambos casos, la página de detalle expone metadatos operativos (por ejemplo, número de paradas y formatos de descarga), así como los atributos incluidos en el conjunto de datos, como la accesibilidad, las tarifas o la geometría de las rutas. Esta información permite verificar de forma rápida el tamaño y la estructura del feed antes de integrarlo en el flujo de procesamiento del simulador.

La propia comunidad GTFS publica recomendaciones de calidad de publicación (persistencia de identificadores, estabilidad de URL de feed, cobertura temporal mínima y buenas prácticas de consistencia), que son relevantes para garantizar explotación automatizada en entornos de producción o experimentación [MobilityData, 2026a]. Además, el validador canónico mantenido por MobilityData se ha convertido en herramienta estándar para control de calidad de feeds estáticos antes de su uso analítico [MobilityData, 2026c]. En este trabajo, GTFS cumple una doble función: por un lado, alimentar la capa de consulta de la red de autobús urbano y, por otro, proporcionar los datos necesarios para construir el grafo multimodal empleado posteriormente por OpenTripPlanner.

2.3. Tecnologías abiertas de enrutado

Una vez descritas las fuentes de datos que permiten representar la red urbana y la oferta de transporte público, la segunda pieza fundamental de un simulador de movilidad es el algoritmo de enrutado. Estos motores permiten transformar la información geoespacial y temporal en itinerarios concretos, así como obtener variables de servicio relevantes, como el tiempo de viaje, la distancia recorrida, los transbordos o las fases de acceso y egreso. En este trabajo se emplean dos tecnologías abiertas con funciones complementarias: OSRM, orientado al cálculo eficiente de rutas sobre red viaria, y OpenTripPlanner, especializado en

planificación multimodal con transporte público.

2.3.1. OSRM

OSRM (*Open Source Routing Machine*) es uno de los motores abiertos más utilizados para cálculo de rutas sobre red viaria OSM, especialmente cuando se necesitan tiempos de respuesta bajos y gran volumen de consultas [?, ?]. Su interés en un simulador de movilidad no está solo en obtener una ruta entre dos puntos, sino en poder generar de forma sistemática atributos de viaje (tiempo, distancia o matrices origen-destino) que después se integran en análisis y modelos predictivos.

La principal fortaleza técnica de OSRM es su enfoque de preprocesado: antes de servir consultas, el sistema transforma el extracto `.osm.pbf`, que es el formato binario comprimido habitualmente utilizado para distribuir datos de OpenStreetMap, en estructuras optimizadas de búsqueda. Este diseño desplaza coste al inicio, pero permite consultas rápidas y estables durante la fase de explotación. Además, OSRM trabaja con perfiles de movilidad diferenciados (por ejemplo, coche, bicicleta y a pie), lo que permite modelar de forma explícita costes de viaje distintos por modo. A nivel interno, OSRM se apoya en algoritmos de aceleración de rutas: MLD (*Multi-Level Dijkstra*), que divide la red en niveles para reducir el espacio de búsqueda en cada consulta, y CH (*Contraction Hierarchies*), que simplifica el grafo con atajos para acelerar rutas largas [Delling et al., 2017, Geisberger et al., 2008].

2.3.2. OpenTripPlanner

OpenTripPlanner (OTP) es un planificador multimodal diseñado para combinar red de calle (OSM) y red de transporte público programado (GTFS) en un único problema de viaje puerta a puerta [OpenTripPlanner Team, 2026b, OpenTripPlanner Team, 2026c]. Su función difiere de la de un motor de enrutado viario convencional, ya que no se limita a calcular el tramo sobre la red de calles, sino que también representa de manera integrada las fases de acceso peatonal, los tiempos de espera, el recorrido en transporte público, los posibles transbordos y el tramo final hasta el destino.

La documentación oficial de OTP describe una arquitectura basada en construcción previa de grafo y posterior explotación por API de planificación. Ese grafo integra simultáneamente geometría de red y programación temporal del servicio, por lo que la consulta de rutas depende de fecha y hora, no solo de coordenadas [OpenTripPlanner Team, 2026a, ?]. Esta dimensión temporal es la base para representar transporte público de forma operativa.

En la salida, OTP devuelve itinerarios descompuestos en tramos o segmentos de viaje,

denominados *legs* en su propia documentación, con información sobre el modo utilizado, los tiempos parciales, el número de transbordos y los metadatos de cada línea. Esta estructura resulta especialmente útil para el análisis multimodal, ya que permite distinguir con precisión las distintas fases del desplazamiento. Por contraste, motores viarios como OSRM están orientados al cálculo rápido de rutas sobre red de calle y no incorporan de forma nativa la lógica propia del transporte público, con horarios, transbordos y secuencias diferenciadas dentro de un mismo itinerario.

2.4. Plataformas propietarias y servicios gestionados

Junto a las tecnologías abiertas de enrutado, el ecosistema actual de movilidad incluye también plataformas propietarias que ofrecen estos mismos servicios mediante API gestionadas. La relevancia industrial de este problema ha favorecido la aparición de soluciones comerciales muy maduras, orientadas tanto al desarrollo de producto como a la integración rápida en aplicaciones web y móviles. Aunque estas plataformas simplifican el despliegue técnico, también introducen condicionantes específicos en términos de coste, dependencia del proveedor y reproducibilidad.

2.4.1. Google Maps Platform

Google Maps Platform se ha convertido en un estándar de facto en los servicios comerciales de localización y navegación orientados al desarrollo de productos digitales. En particular, *Routes API* ofrece cálculo de rutas multimodo, metadatos de viaje y un ecosistema de integración especialmente maduro para aplicaciones web, móviles y, de forma destacada, entornos basados en Android [Google, 2026b].

Para proyectos de desarrollo de producto, este modelo reduce de forma notable la complejidad operativa, ya que no exige desplegar ni mantener infraestructura propia de enrutado y permite acceder a funcionalidades avanzadas mediante API. Esta característica ha favorecido su adopción en aplicaciones comerciales, especialmente en entornos móviles y ecosistemas integrados con servicios de Google. No obstante, en contextos de investigación aplicada o en escenarios con un volumen elevado de simulaciones, aparecen limitaciones relevantes: por un lado, el coste económico crece con el número de peticiones y con el tipo de servicio consumido; por otro, la ejecución queda condicionada por cuotas de uso, políticas del proveedor y disponibilidad externa de la plataforma [Google, 2026a]. Estas restricciones reducen su idoneidad cuando se busca trazabilidad completa, control experimental y

escalabilidad económica predecible.

2.4.2. Otras plataformas de referencia

Además de Google Maps Platform, el ecosistema actual incluye otras alternativas consolidadas de consumo por API, entre ellas Mapbox [Mapbox, 2026] o TomTom [?]. Estas plataformas ofrecen catálogos amplios (rutas, matrices, navegación, tráfico y servicios avanzados según proveedor), y comparten una lógica común de uso gestionado mediante credenciales, límites de consumo y planes de servicio.

También existen servicios intermedios como openrouteservice, que proporcionan API pública y, al mismo tiempo, opciones de despliegue propio [HeiGIT gmbH, 2026, openrouteservice, 2026]. Este tipo de solución puede ser útil como puente entre experimentación rápida y control de infraestructura, aunque con sus propias restricciones de uso en modalidad pública.

La Tabla 2.1 resume esta comparación y la decisión tecnológica adoptada en el proyecto. Para un simulador académico orientado a experimentación, trazabilidad y ejecución repetida de escenarios, la combinación OSRM + OTP en infraestructura propia ofrece un equilibrio más adecuado que una API comercial gestionada, aunque implique una mayor complejidad de despliegue.

Tabla 2.1: Comparación resumida de las soluciones de enrutado consideradas en este trabajo. Elaboración propia a partir de [?, OpenTripPlanner Team, 2026b, Google, 2026b, Google, 2026a].

Solución	Tipo de red	Coste de uso	Adecuación al TFM
OSRM	Red viaria OSM, perfiles por modo	Infraestructura propia	Muy adecuado para consultas masivas y control experimental
OTP	Red multimodal OSM + GTFS	Infraestructura propia	Muy adecuado para transporte público y análisis puerta a puerta
Google Maps Platform	API comercial multimodo gestionada	Pago por petición y cuotas	Útil como referencia industrial, menos adecuado para reproducibilidad

2.5. Modelado de elección modal

Una vez descrita la infraestructura tecnológica que permite representar la red y calcular itinerarios, es necesario incorporar una capa de modelado que permita analizar el comportamiento de los viajeros. En un simulador de movilidad, no basta con conocer qué alternativas existen para un desplazamiento determinado, sino que también es necesario estimar cuál de ellas es más probable que sea elegida en función de sus atributos de servicio y de las características del usuario. Esta cuestión se aborda habitualmente mediante modelos de elección discreta y, más recientemente, mediante técnicas de aprendizaje automático aplicadas a la predicción de la elección modal [?, Martín-Baos, 2023].

En la literatura reciente, una de las referencias más utilizadas para este tipo de problemas es el dataset London Passenger Mode Choice (LPMC), construido a partir de viajes observados en Londres y enriquecido con variables de nivel de servicio para distintos modos de transporte [Hillel et al., 2018, Hillel, 2019]. Su interés en este trabajo no está en el caso de estudio concreto, sino en que proporciona una base metodológica y experimental muy próxima al problema abordado en este TFM.

2.5.1. Modelos de utilidad aleatoria

Este problema se ha abordado tradicionalmente mediante los denominados modelos de utilidad aleatoria (*Random Utility Models*, RUM), ampliamente utilizados en transporte para analizar decisiones de elección discreta, como la elección del modo de transporte. En este enfoque, el decisor elige una alternativa dentro de un conjunto de opciones mutuamente excluyentes, por ejemplo caminar, usar la bicicleta, utilizar transporte público o desplazarse en coche [?, McFadden, 1974].

La idea central de estos modelos es que cada viajero asocia una utilidad a cada alternativa disponible dentro de su conjunto de elección, y se asume que elegirá aquella que le proporcione una mayor utilidad. Sin embargo, dicha utilidad no puede observarse de forma completa, ya que parte de los factores que influyen en la decisión sí pueden ser medidos por el analista, mientras que otros permanecen ocultos o no observados. Por ello, la utilidad de una alternativa suele descomponerse en un término determinista y un término aleatorio:

$$U_{in} = V_{in} + \varepsilon_{in} \quad (2.1)$$

donde U_{in} representa la utilidad total que el individuo n asocia a la alternativa i , V_{in} es la componente sistemática o determinista de dicha utilidad, y ε_{in} recoge los factores no

observados que también influyen en la decisión.

Bajo este planteamiento, el viajero se considera racional en el sentido de que selecciona la alternativa que maximiza su utilidad. En consecuencia, la elección observada puede expresarse en términos probabilísticos como la probabilidad de que la utilidad de una alternativa sea mayor o igual que la del resto de alternativas disponibles:

$$P_{in} = P(U_{in} \geq U_{jn} \forall j \in C) \quad (2.2)$$

donde C representa el conjunto de alternativas consideradas. De este modo, los RUM permiten traducir un problema de comportamiento individual en un modelo probabilístico de elección, capaz de relacionar atributos del viaje, características del individuo y probabilidad de seleccionar cada modo de transporte.

2.5.2. Modelo Logit Multinomial

Entre los modelos derivados del enfoque RUM, el más extendido en el análisis de elección modal es el Modelo Logit Multinomial (*Multinomial Logit*, MNL). Su popularidad se debe a que ofrece una formulación matemática sencilla, una interpretación clara de los parámetros y una estimación relativamente eficiente desde el punto de vista computacional. En el contexto del transporte, el MNL ha sido utilizado de forma recurrente para estudiar cómo variables como el tiempo de viaje, el coste, los transbordos o determinadas características sociodemográficas influyen en la probabilidad de elegir un modo de transporte frente a otro [McFadden, 1974].

El modelo MNL se obtiene al asumir que el término aleatorio de la utilidad sigue una distribución Gumbel independiente e idénticamente distribuida para todas las alternativas. Bajo esta hipótesis, la probabilidad de que el individuo n elija la alternativa i puede expresarse de la siguiente forma:

$$P_{in} = \frac{\exp(V_{in})}{\sum_{j=1}^I \exp(V_{jn})} \quad (2.3)$$

donde I es el número total de alternativas disponibles y V_{in} representa la utilidad sistemática de la alternativa i para el individuo n . Esta expresión da lugar a una función de probabilidad de tipo sigmoideal, de manera que pequeñas variaciones en la utilidad representativa tienen mayor efecto cuando la probabilidad de elección se encuentra en valores intermedios que cuando una alternativa ya es claramente dominante o claramente poco atractiva. Esta propiedad resulta especialmente útil para interpretar cambios marginales en los atributos del servicio, por ejemplo ante mejoras en tiempo o coste de una determinada alternativa.

La Figura 2.1 ilustra de forma esquemática esta forma sigmoïdal de la probabilidad de elecci3n en el modelo MNL.

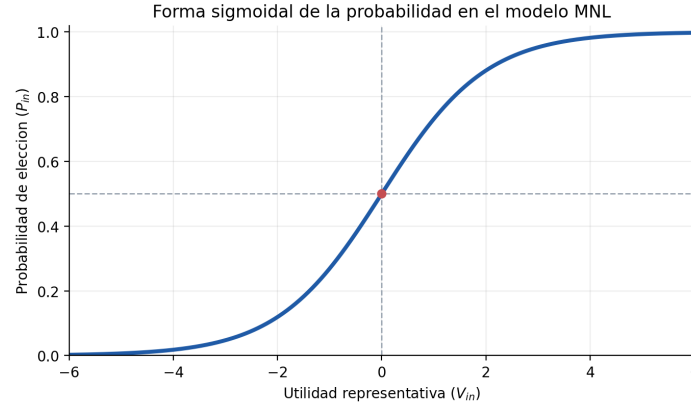


Figura 2.1: Forma sigmoïdal ilustrativa de la probabilidad de elecci3n en el modelo MNL. Elaboraci3n propia.

En la pr3ctica, la componente determinista de la utilidad suele especificarse como una combinaci3n lineal de atributos observables:

$$V_{in} = \beta_{i0} + \sum_k \beta_{ik} x_{ink} \quad (2.4)$$

donde x_{ink} representa el valor del atributo k asociado a la alternativa i para el individuo n , β_{ik} es el par3metro que mide la influencia de dicho atributo y β_{i0} es una constante especı́fica de la alternativa. Esta formulaci3n permite cuantificar el efecto relativo de variables como el tiempo de viaje, el coste monetario, el n3mero de transbordos o el acceso peatonal, entre otras. En t3rminos interpretativos, los par3metros estimados indican c3mo varı́a la utilidad de cada modo cuando cambian sus atributos observables.

La estimaci3n de los par3metros del modelo se realiza habitualmente mediante m3xima verosimilitud. Para ello, se define una funci3n de verosimilitud que mide la probabilidad de observar las elecciones registradas en la muestra dadas unas determinadas estimaciones de los par3metros. El objetivo del proceso de ajuste consiste en encontrar los valores de β que maximizan dicha verosimilitud, o de forma equivalente, que maximizan la log-verosimilitud del modelo. Este procedimiento constituye la base cl3sica de estimaci3n de modelos de elecci3n discreta y permite obtener un modelo interpretable y coherente con la teorı́a del comportamiento del viajero.

En este TFM, el modelo MNL resulta relevante como referencia metodológica, ya que proporciona una base teórica sólida para representar la elección modal a partir de atributos del desplazamiento. Además, sirve como punto de comparación frente a enfoques más flexibles basados en aprendizaje automático, que se describen en los apartados siguientes.

Una limitación conocida del modelo MNL es la hipótesis de independencia de alternativas irrelevantes (*Independence of Irrelevant Alternatives*, IIA), derivada de la suposición de errores independientes e idénticamente distribuidos. Esta propiedad implica que la relación de sustitución entre dos alternativas no depende de la presencia o de las características del resto de modos, lo cual puede resultar restrictivo en problemas de movilidad donde algunas opciones están más correlacionadas entre sí que otras [McFadden, 1974, ?]. Aun así, el MNL sigue siendo el punto de partida más habitual en la literatura por su equilibrio entre simplicidad, interpretabilidad y capacidad descriptiva.

2.5.3. Limitaciones del enfoque clásico

A pesar de su amplia utilización en el análisis de elección modal, los modelos basados en utilidad aleatoria presentan ciertas limitaciones cuando se aplican a problemas complejos de movilidad. En primer lugar, requieren especificar de forma explícita la forma funcional de la utilidad sistemática, lo que implica que el analista debe decidir previamente qué variables incluir y cómo se relacionan con la utilidad de cada alternativa. En muchos casos, esta especificación se realiza mediante combinaciones lineales de atributos del viaje, lo que puede resultar restrictivo cuando las relaciones reales entre variables son no lineales o presentan interacciones complejas [Martín-Baos, 2023, p. 20].

Otra limitación relevante es que los modelos clásicos suelen asumir estructuras relativamente simples en la distribución de los términos de error. En el caso del modelo Logit Multinomial, por ejemplo, la hipótesis de independencia de alternativas irrelevantes puede no reflejar adecuadamente situaciones en las que algunas alternativas presentan correlaciones más fuertes entre sí, como ocurre frecuentemente entre distintos modos de transporte público o entre modos motorizados.

Además, desde un punto de vista práctico, la especificación de modelos econométricos puede requerir un proceso iterativo de selección de variables, transformación de atributos y validación de supuestos teóricos. Aunque este enfoque proporciona modelos interpretables y coherentes con la teoría del comportamiento del viajero, puede resultar menos flexible cuando se trabaja con conjuntos de datos de alta dimensionalidad o con relaciones complejas entre variables.

2.5.4. Aprendizaje automático para elección modal

Con el aumento de la disponibilidad de datos y de la capacidad computacional, en los últimos años han ganado relevancia los enfoques basados en aprendizaje automático para el análisis de elección modal. Desde esta perspectiva, la elección de modo puede formularse también como un problema de clasificación supervisada, en el que el objetivo consiste en predecir la alternativa elegida a partir de un conjunto de variables explicativas relacionadas con el viaje y con las características del individuo.

A diferencia de los modelos clásicos de elección discreta, muchos algoritmos de aprendizaje automático no requieren especificar de forma explícita la forma funcional de la relación entre variables. En su lugar, aprenden patrones directamente a partir de los datos de entrenamiento, lo que les permite capturar relaciones no lineales, interacciones complejas entre atributos y estructuras de decisión difíciles de representar mediante modelos lineales tradicionales.

Entre las técnicas más utilizadas en este contexto destacan los métodos basados en árboles de decisión y los modelos de conjunto (*ensemble models*) derivados de ellos, como Random Forest o Gradient Boosting Decision Trees, así como los modelos basados en redes neuronales profundas. Estos enfoques han mostrado un rendimiento predictivo competitivo en numerosos problemas de clasificación tabular, incluyendo aplicaciones en transporte y elección modal.

En este trabajo se adopta esta perspectiva complementaria: el modelo Logit Multinomial se considera como referencia metodológica clásica, mientras que distintos modelos de aprendizaje automático se utilizan para explorar alternativas con mayor capacidad predictiva. En particular, la arquitectura del simulador se ha diseñado para permitir la integración de diferentes modelos de clasificación, entre ellos Random Forest, Gradient Boosting Decision Trees y redes neuronales profundas, de forma que puedan compararse sus resultados en el contexto del problema de elección modal analizado. Esta comparación resulta coherente con la evidencia reciente disponible para datasets reales de elección modal, entre ellos LPMC [Hillel et al., 2018].

2.5.5. Random Forests

Random Forest es un método de aprendizaje automático basado en la combinación de múltiples árboles de decisión entrenados sobre subconjuntos distintos de los datos. El modelo fue propuesto por Breiman [Breiman, 2001] como una extensión de los árboles de clasificación tradicionales, con el objetivo de mejorar su estabilidad y capacidad predictiva. En

diversas revisiones de la literatura, este tipo de modelos aparece además como una de las familias más utilizadas en comparativas recientes de elección modal.

La idea fundamental consiste en entrenar un gran número de árboles de decisión sobre diferentes subconjuntos de los datos de entrenamiento. Cada árbol se construye utilizando una muestra aleatoria del conjunto de datos original (técnica conocida como *bootstrap sampling*) y, además, en cada división del árbol se considera únicamente un subconjunto aleatorio de variables explicativas. Este doble proceso de aleatorización permite reducir la correlación entre árboles individuales y, en consecuencia, mejorar el rendimiento del modelo agregado.

En problemas de clasificación, como el de elección modal, cada árbol produce una predicción independiente y la decisión final se obtiene mediante votación mayoritaria entre todos los árboles del bosque. Este mecanismo permite capturar relaciones no lineales entre variables y manejar de forma robusta conjuntos de datos con múltiples atributos.

En el ámbito del transporte, los modelos Random Forest se han utilizado en distintos trabajos para predecir la elección de modo de transporte, ya que ofrecen un buen equilibrio entre capacidad predictiva, robustez frente al sobreajuste y relativa facilidad de interpretación mediante medidas de importancia de variables. No obstante, la evidencia comparativa reciente sugiere que su rendimiento puede quedar por debajo del de otros métodos más potentes de boosting en algunos conjuntos de datos reales [José Ángel Martín-Baos, 2023].

2.5.6. Gradient Boosting Decision Trees

Los modelos basados en *Gradient Boosting Decision Trees* (GBDT) constituyen otra familia de métodos de aprendizaje automático ampliamente utilizados en problemas de clasificación con datos tabulares. A diferencia de los métodos basados en bagging, como Random Forest, el enfoque de boosting construye los árboles de decisión de forma secuencial, de modo que cada nuevo árbol intenta corregir los errores cometidos por el conjunto de árboles anteriores [Friedman, 2001].

El proceso comienza entrenando un primer árbol de decisión sencillo sobre los datos de entrenamiento. A continuación, se construyen árboles adicionales que se ajustan sobre los residuos o errores del modelo previo. De este modo, el modelo final se obtiene como la suma ponderada de múltiples árboles débiles, cada uno de los cuales contribuye a mejorar gradualmente la predicción global.

Entre las implementaciones más conocidas de este enfoque se encuentra XGBoost (*Extreme Gradient Boosting*), una biblioteca optimizada que introduce mejoras en eficiencia computacional, regularización del modelo y manejo de grandes conjuntos de datos

[Chen and Guestrin, 2016]. Gracias a estas características, XGBoost se ha convertido en una de las técnicas más utilizadas en problemas de aprendizaje supervisado con datos estructurados y ha mostrado un comportamiento especialmente competitivo en comparativas recientes de elección modal.

En el contexto de este TFM, los modelos basados en Gradient Boosting resultan especialmente relevantes, ya que permiten capturar relaciones no lineales entre los atributos del viaje y la elección modal. Además, su buen rendimiento predictivo y su flexibilidad para manejar distintos tipos de variables los convierten en una herramienta adecuada para integrarse en el simulador de movilidad desarrollado en este trabajo.

2.5.7. Redes Neuronales Profundas

Las redes neuronales profundas (*Deep Neural Networks*, DNN) constituyen otra familia de modelos de aprendizaje automático que ha experimentado un crecimiento significativo en los últimos años. Estas técnicas se basan en arquitecturas formadas por múltiples capas de neuronas artificiales que transforman progresivamente las variables de entrada mediante combinaciones lineales y funciones de activación no lineales [Goodfellow et al., 2016].

A diferencia de los modelos basados en árboles, las redes neuronales permiten aprender representaciones internas de los datos a través de varias capas ocultas, lo que facilita capturar relaciones complejas entre variables. Esta capacidad de modelar interacciones no lineales ha motivado su aplicación en numerosos problemas de predicción y clasificación, incluyendo el análisis del comportamiento de los viajeros.

En problemas de elección modal, las redes neuronales profundas pueden utilizarse para estimar la probabilidad de seleccionar cada modo de transporte a partir de atributos del desplazamiento y características sociodemográficas del usuario. Aunque estos modelos suelen requerir mayor volumen de datos y un proceso de ajuste más cuidadoso que los métodos basados en árboles, también ofrecen una elevada flexibilidad para capturar patrones complejos presentes en los datos.

Los tres enfoques descritos en esta sección (Random Forest, Gradient Boosting y redes neuronales profundas) se evalúan de forma comparativa sobre el dataset LPMC. Su entrenamiento, ajuste e integración en el simulador de movilidad se detallan en el capítulo 5.

3. Metodología

Este capítulo recoge la gestión del proyecto con un enfoque SCRUM adaptado a un TFM individual. Se busca dejar trazabilidad clara de qué se planificó, qué se ejecutó, qué bloqueos aparecieron y qué decisiones técnicas se tomaron en cada iteración.

3.1. SCRUM adaptado a un desarrollo unipersonal

SCRUM es una metodología ágil de carácter iterativo e incremental, orientada a organizar el trabajo en ciclos cortos, priorizar entregas de valor y revisar de forma continua el avance del producto [?]. Aunque su formulación original está pensada para equipos, sus principios también pueden adaptarse a proyectos individuales cuando se busca mantener planificación, trazabilidad y capacidad de reacción ante cambios.

En este trabajo se ha utilizado una adaptación práctica de SCRUM, no como aplicación estricta del marco en todos sus elementos formales, sino como guía para estructurar el desarrollo del simulador, registrar decisiones y ordenar la evolución del proyecto. Esta adaptación ha resultado útil por tres motivos. En primer lugar, porque se han combinado tareas heterogéneas, incluyendo desarrollo backend, frontend, integración de motores de enrutado, tratamiento de datos, entrenamiento de modelos y redacción de memoria. En segundo lugar, porque el alcance real del proyecto ha ido evolucionando a medida que se validaban soluciones técnicas y se detectaban bloqueos. Por último, porque el seguimiento periódico con el tutor ha funcionado como mecanismo natural de revisión y reorientación.

Desde el punto de vista organizativo, la Guía Scrum 2020 define un *Scrum Team* compuesto por tres responsabilidades principales: *Product Owner*, *Scrum Master* y *Developers* [?]. En un trabajo unipersonal esta separación no puede reproducirse de forma literal, por lo que se ha reinterpretado del siguiente modo:

-
- **Product Owner:** el tutor académico ha asumido de forma aproximada esta función, al validar prioridades, orientar el alcance funcional del prototipo y revisar la utilidad de los entregables desde el punto de vista del objetivo del TFM.
 - **Scrum Master:** esta responsabilidad ha sido asumida por el propio autor del trabajo, en la medida en que ha organizado iteraciones, registrado bloqueos, mantenido la trazabilidad del progreso y ajustado la planificación cuando han surgido incidencias técnicas o cambios de enfoque.
 - **Developers:** el desarrollo técnico también ha recaído íntegramente en el autor, responsable de la implementación del simulador, la integración de servicios, el entrenamiento de modelos y la documentación de resultados.

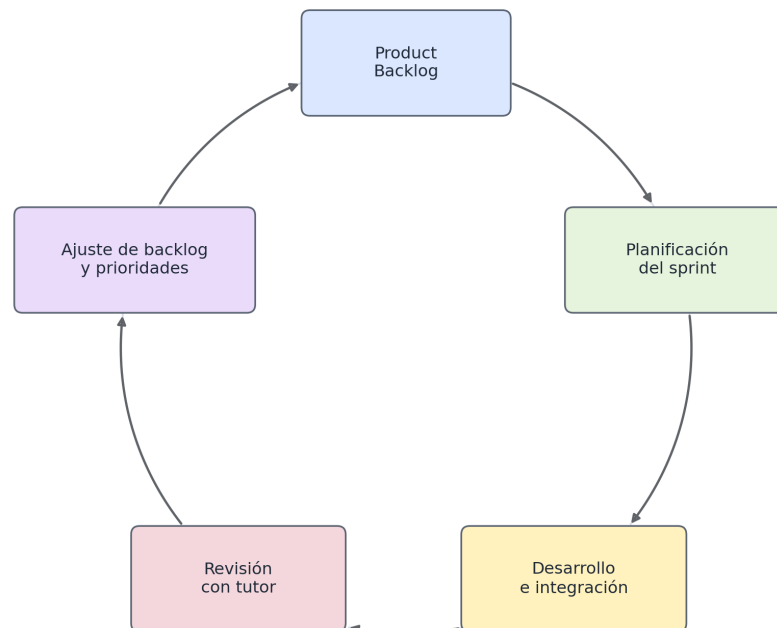
En consecuencia, varias responsabilidades propias de un equipo Scrum han quedado concentradas en una única persona, mientras que el tutor ha actuado como agente externo de supervisión, validación y priorización. Esta circunstancia introduce diferencias claras respecto a un SCRUM de equipo, pero no invalida su uso como marco ligero de gestión. En este contexto, lo más relevante no ha sido reproducir todas las ceremonias de forma estricta, sino conservar los elementos que aportan valor real al proyecto: backlog priorizado, trabajo por iteraciones, revisión periódica y registro explícito de decisiones.

La metodología seguida en este trabajo se ha apoyado en los siguientes principios:

- dividir el trabajo en sprints con objetivos acotados y entregables identificables;
- mantener un *Product Backlog* vivo, revisado conforme avanzaba el proyecto;
- registrar en cada iteración qué se planificó, qué se completó y qué problemas aparecieron;
- utilizar las reuniones con el tutor como mecanismo de revisión y re-priorización;
- orientar cada sprint a la obtención de un incremento funcional o documental del proyecto.

En cuanto al seguimiento, las reuniones con el tutor comenzaron a finales de 2025, aproximadamente entre octubre y noviembre, con una periodicidad quincenal, normalmente los martes. Una vez finalizado el primer cuatrimestre y superada la carga principal de exámenes, la cadencia pasó a ser semanal a partir de febrero y marzo de 2026, con el objetivo de acelerar el ritmo de desarrollo y acotar con mayor precisión el trabajo asociado a cada sprint. Este cambio de frecuencia reforzó el carácter iterativo del proyecto y facilitó una validación más cercana del backlog y de los entregables.

La Figura 3.1 resume visualmente este ciclo de trabajo adaptado a un desarrollo unipersonal, mostrando la relación entre backlog, planificación, ejecución y revisión periódica con el tutor.



SCRUM adaptado a desarrollo unipersonal

Figura 3.1: Esquema del ciclo de trabajo SCRUM adaptado al desarrollo unipersonal del proyecto. Elaboración propia.

Esta metodología ha encajado bien con la naturaleza del proyecto. Por una parte, ha permitido avanzar de manera incremental desde un prototipo inicial de rutas hasta un MVP funcional con integración de OSRM, OTP, GTFS y un modelo de elección modal. Por otra, ha facilitado documentar de forma ordenada tanto el trabajo ya completado como las líneas de evolución previstas, incluyendo mejoras visuales, ampliación de modelos y posibles análisis adicionales de simulación. A partir de este marco general, en los apartados siguientes se presenta el backlog del proyecto y el histórico de sprints ejecutados.

3.2. Product Backlog

3.2.1. Backlog inicial

El *Product Backlog* se definió como una lista viva de trabajo, organizada en grandes bloques funcionales y técnicos. En lugar de partir de un conjunto cerrado de tareas detalladas desde el inicio, se optó por un backlog de alto nivel que pudiera refinarse a medida que se validaban decisiones de diseño y se consolidaba el MVP del simulador.

En una primera aproximación, los elementos principales del backlog fueron los siguientes:

1. Definir una arquitectura base cliente-servidor para desacoplar frontend, backend y servicios externos.
2. Construir un prototipo web funcional con mapa interactivo, selección de origen-destino y visualización de rutas.
3. Integrar OSRM en local con perfiles diferenciados para coche, bicicleta y desplazamiento a pie.
4. Incorporar transporte público urbano mediante GTFS de Toledo y OpenTripPlanner.
5. Diseñar una API propia que unifique consultas de rutas, exploración GTFS e inferencia modal.
6. Preparar una pipeline reproducible de datos para el dataset LPMC.
7. Entrenar y validar un primer modelo de elección modal utilizable dentro del simulador.
8. Integrar el modelo de inferencia en backend y frontend de forma modular.
9. Mejorar la interfaz, la experiencia de usuario y la capacidad de análisis visual del sistema.
10. Documentar la solución, registrar la evolución del proyecto y redactar la memoria final.

Conforme avanzó el trabajo, este backlog se fue refinando en tareas más concretas. Algunos elementos inicialmente planteados como trabajo futuro pasaron a formar parte del desarrollo principal, mientras que otros quedaron aplazados para iteraciones posteriores. A fecha de redacción de esta memoria, la parte nuclear del simulador puede considerarse ya implementada, por lo que el backlog actual se concentra principalmente en consolidación documental, mejoras visuales y ampliaciones funcionales sobre una base ya operativa.

Para facilitar su seguimiento, los elementos del backlog se agruparon en cuatro bloques:

- **Bloque funcional:** interfaz web, selección de viajes, cálculo de rutas, visualización de resultados y cuadro de información.
- **Bloque de integración de servicios:** despliegue y conexión de OSRM, OTP y GTFS dentro de una arquitectura reproducible.
- **Bloque de inteligencia artificial:** preprocesado del dataset LPMC, entrenamiento de modelos, validación e integración de inferencia.
- **Bloque documental y de cierre:** trazabilidad del proyecto, redacción de memoria, mejora de la presentación y preparación de resultados.

3.2.2. Priorización de items

La priorización del backlog no se realizó únicamente por orden técnico de implementación, sino por valor aportado al objetivo general del proyecto. En consecuencia, se siguió el siguiente criterio:

- **Prioridad alta:** elementos necesarios para disponer de un simulador funcional extremo a extremo, incluyendo mapa interactivo, rutas por modo, integración de transporte público y una primera versión operativa de la inferencia modal.
- **Prioridad media:** mejoras que aumentan la usabilidad, la claridad visual o la trazabilidad del sistema, pero que no bloquean el funcionamiento básico del MVP.
- **Prioridad evolutiva:** ampliaciones previstas una vez estabilizado el núcleo del sistema, como el entrenamiento de modelos adicionales, la mejora de análisis de escenarios o la incorporación de nuevas visualizaciones y métricas.

Este criterio explica el orden seguido durante el desarrollo. Primero se priorizó la infraestructura mínima para obtener rutas y visualizar resultados reales sobre el mapa; después se incorporó el bloque de modelado e inferencia; y finalmente se abordaron tareas de consolidación, mejora y documentación. De esta forma, cada sprint pudo orientarse a producir incrementos útiles sobre una base ya validada, en lugar de avanzar en paralelo sobre demasiados frentes abiertos.

3.3. Histórico de sprints ejecutados

En este apartado se resume la secuencia principal de trabajo seguida durante el desarrollo del proyecto. Dado que el TFM no arrancó con una planificación completamente cerrada

y que parte de la trazabilidad se ha reconstruido a partir de versiones funcionales, reuniones de seguimiento y artefactos generados, los periodos y esfuerzos indicados deben interpretarse como estimaciones razonables. Aun así, este histórico permite mostrar con claridad cómo evolucionó el proyecto desde la validación inicial de tecnologías hasta la integración final del simulador.

La Figura 3.2 muestra un diagrama de Gantt simplificado de los sprints ejecutados durante el desarrollo del proyecto. El diagrama refleja tanto la secuencia principal del desarrollo como la existencia de trabajo en paralelo, especialmente entre las fases de modelado y diseño web y en la redacción de la memoria.

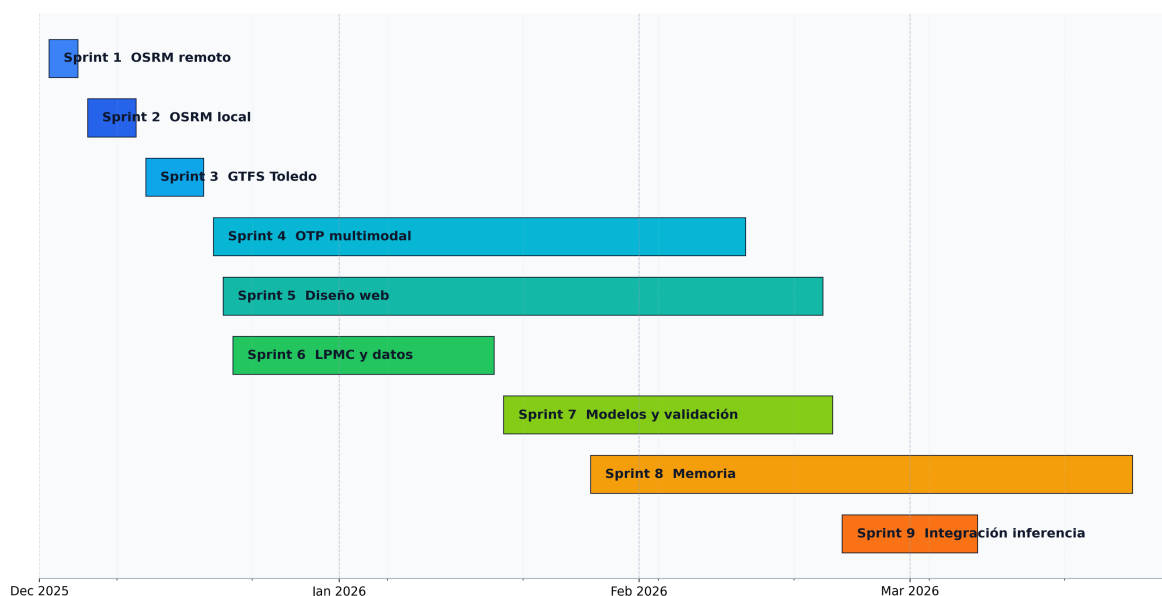


Figura 3.2: Cronograma aproximado de los sprints ejecutados durante el desarrollo del TFM, representado como diagrama de Gantt simplificado. Elaboración propia.

3.3.1. Sprint 1: Validación inicial de OSRM mediante servicios remotos

Fechas: del 2 al 5 de diciembre de 2025.

Esfuerzo estimado: 8–12 horas.

Este sprint tuvo como objetivo investigar distintas herramientas de enrutado y, en particular, validar la viabilidad de OSRM dentro del proyecto mediante llamadas remotas desde una primera arquitectura cliente-servidor. El propósito no era todavía disponer de una so-

lución definitiva, sino comprobar que el flujo completo entre mapa, backend y cálculo de rutas resultaba técnicamente factible.

El principal resultado de esta iteración fue la construcción de un primer prototipo funcional y la identificación temprana de las limitaciones de los servicios remotos utilizados para las pruebas iniciales. Esta validación permitió confirmar el interés de OSRM como base del simulador, pero también puso de manifiesto la necesidad de avanzar hacia una infraestructura propia para garantizar comparabilidad real entre modos y mayor control experimental.

3.3.2. Sprint 2: Despliegue del servicio OSRM local multi-perfil

Fechas: del 6 al 11 de diciembre de 2025.

Esfuerzo estimado: 12–16 horas.

Una vez validado el uso de OSRM para este trabajo, este sprint se orientó al despliegue de una instancia local que permitiera obtener rutas diferenciadas para distintos modos de desplazamiento y, al mismo tiempo, mantener un control total sobre la infraestructura. Este cambio era necesario para asegurar reproducibilidad, trazabilidad y estabilidad en las comparaciones realizadas por el simulador.

Como resultado, el proyecto dejó de depender de servicios públicos externos y pasó a apoyarse en una base de cálculo local sobre la que posteriormente se construirían tanto la comparación modal como la integración con el resto de componentes del sistema.

3.3.3. Sprint 3: Incorporación del GTFS urbano de Toledo

Fechas: del 12 al 18 de diciembre de 2025.

Esfuerzo estimado: 10–14 horas.

El objetivo de esta iteración fue incorporar la red de transporte público urbano de Toledo al simulador a partir del feed GTFS descargado desde el Punto de Acceso Nacional (NAP) del Ministerio de Transportes. Para ello se desarrolló el módulo de lectura `gtfs_loader.py`, que parsea y mantiene en memoria la información de paradas, rutas, viajes y horarios, y se implementaron cuatro endpoints en el router `/api/gtfs`: listado de paradas con filtro opcional por bounding box, listado de líneas de la red, detalle de ruta con paradas ordenadas y trazado geométrico, y resumen de horarios por fecha.

En la capa de visualización se añadieron las paradas como marcadores circulares en el mapa con popup interactivo, desde el que el usuario puede consultar qué líneas dan servicio en cada parada. Este sprint no incluyó planificación de itinerarios, que se abordó en el sprint siguiente. Su resultado principal fue disponer de una representación operativa completa de

la red de bus urbano de Toledo, desacoplada del grafo OTP, lo que permite consultar la red aunque el planificador no esté disponible.

3.3.4. Sprint 4: Integración de OpenTripPlanner para rutas multimodales

Fechas: del 19 de diciembre de 2025 al 12 de febrero de 2026.

Esfuerzo estimado: 16–24 horas.

Tras la incorporación del GTFS, este sprint se centró en habilitar el cálculo de rutas de transporte público puerta a puerta mediante OpenTripPlanner. El objetivo era pasar de una visualización estática de red a una capacidad real de planificación multimodal, incorporando la combinación entre red peatonal y servicio de autobús urbano.

Esta fase supuso un salto cualitativo dentro del proyecto, ya que permitió integrar en una misma aplicación los modos viarios y el modo tránsito, haciendo posible una comparación más completa entre alternativas de viaje en el contexto urbano de Toledo.

3.3.5. Sprint 5: Diseño de la interfaz web y codificación visual por modo

Fechas: del 20 de diciembre de 2025 al 20 de febrero de 2026.

Esfuerzo estimado: 8–12 horas.

Este sprint se orientó a consolidar la experiencia de usuario y la legibilidad comparada de resultados. Las principales decisiones de diseño adoptadas fueron: codificación visual de rutas por modo (azul sólido para conducción, verde sólido para bicicleta y gris discontinuo para desplazamiento a pie); visualización de segmentos OTP diferenciada entre tramos en vehículo, representados en naranja, y tramos a pie, con la misma trama discontinua que el modo peatonal; cuatro basemaps seleccionables por el usuario (carto-light, OpenStreet-Map estándar, OpenTopoMap y satélite Esri); y marcadores SVG personalizados para origen y destino con iconografía diferenciada.

Se consolidó también el uso de TanStack Query para las peticiones al backend, que proporciona caché de respuestas, gestión declarativa de estados de carga y reintento automático en caso de error. Los segmentos de transporte público se restringieron a modo *transit*, evitando solapamientos visuales con las rutas viarias. El resultado fue una interfaz directamente utilizable como herramienta de análisis, no solo como demostración técnica del sistema de enrutado.

3.3.6. Sprint 6: Diseño de un modelo de clasificación de modo de viaje y procesamiento de datos

Fechas: del 21 de diciembre de 2025 al 17 de enero de 2026.

Esfuerzo estimado: 12–18 horas.

Este sprint tuvo como finalidad preparar la base metodológica del módulo de inteligencia artificial del proyecto. Para ello se abordó el estudio del dataset LPMC, la revisión de trabajos de referencia y el diseño del proceso de preparación de datos necesario para entrenar modelos de clasificación de modo de viaje con un criterio reproducible.

El trabajo realizado en esta fase no se orientó todavía a la integración en el simulador, sino a establecer una base sólida de modelado sobre la que poder entrenar, comparar y validar distintos enfoques en iteraciones posteriores.

3.3.7. Sprint 7: Entrenamiento y validación de los modelos de clasificación

Fechas: del 18 de enero al 21 de febrero de 2026.

Esfuerzo estimado: 14–20 horas.

Una vez definida la base de datos y el proceso de preparación, este sprint se centró en entrenar y validar los primeros modelos de clasificación de modo de viaje. El objetivo fue disponer de una referencia cuantitativa útil, suficientemente estable como para servir de punto de partida a la integración del módulo de inferencia dentro del simulador.

Como resultado, el proyecto avanzó desde una fase exploratoria sobre datos hacia una fase claramente aplicada, en la que ya era posible valorar el rendimiento de los modelos y decidir cuáles podían incorporarse al sistema de forma operativa.

3.3.8. Sprint 8: Escritura de la memoria y consolidación metodológica

Fechas: del 27 de enero al 24 de marzo de 2026, en paralelo a varios sprints técnicos.

Esfuerzo estimado: 10–14 horas en su arranque, ampliado de forma continua durante la redacción final.

La escritura de la memoria no se desarrolló como una actividad aislada al final del proyecto, sino como una línea de trabajo paralela que fue ganando peso a medida que el simulador y el bloque de modelado alcanzaban mayor madurez. El objetivo de este sprint fue consolidar la estructura documental del TFM, fijar el enfoque metodológico y dejar trazabilidad suficiente de las decisiones adoptadas a lo largo del desarrollo.

Su carácter transversal hace que este sprint deba interpretarse como una actividad de

acompañamiento al resto del proyecto. En la práctica, sirvió para ordenar el material generado, depurar la narrativa técnica y conectar la evolución funcional del sistema con la justificación académica del trabajo.

3.3.9. Sprint 9: Integración del módulo de inferencia en el simulador

Fechas: del 22 de febrero al 8 de marzo de 2026.

Esfuerzo estimado: 14–20 horas.

El objetivo de esta iteración fue integrar el módulo de inferencia modal dentro del simulador ya construido, conectando el bloque de modelado con la aplicación web y con la lógica general del backend. Con ello se buscaba que el sistema dejara de ser únicamente un comparador de rutas para incorporar también una capa predictiva sobre la elección de modo de viaje.

Esta integración representó la convergencia de las dos grandes líneas del TFM: por un lado, la construcción del simulador de movilidad urbana; por otro, el diseño y validación de modelos de clasificación. A partir de este punto, el proyecto quedó configurado como una aplicación final capaz de combinar cálculo de rutas, visualización cartográfica e inferencia modal en un mismo entorno de trabajo.

3.4. Cierre del ciclo de desarrollo

La secuencia de sprints descrita en este capítulo llevó el proyecto desde una primera validación de tecnologías de enrutado hasta un simulador funcional extremo a extremo, con integración de rutas viarias, transporte público multimodal y predicción de elección modal mediante aprendizaje automático. Cada iteración aportó un incremento concreto sobre la base anterior: el prototipo inicial de rutas se amplió con un servicio local multiperfil, se integró la red de transporte público de Toledo, se incorporó la planificación multimodal con OTP, se consolidó la interfaz y, finalmente, se conectó el módulo de inferencia con el resto del sistema.

La arquitectura resultante y las decisiones técnicas adoptadas a lo largo de este proceso se describen en el capítulo 4. Los detalles de implementación de cada componente se recogen en el capítulo 5.

4. Arquitectura del sistema

Este capítulo describe la organización estructural del simulador: los componentes que lo forman, cómo se relacionan entre sí y las decisiones de diseño que condicionan esa organización. El detalle de implementación de cada componente se desarrolla en el capítulo 5.

El sistema está compuesto por cinco bloques funcionales: una interfaz web, un backend de orquestación, tres instancias del motor de enrutado viario OSRM, un planificador de transporte público basado en OpenTripPlanner y un módulo de inferencia de elección modal. Todos ellos se despliegan como contenedores Docker orquestados mediante Docker Compose.

4.1. Visión general

La Figura 4.1 muestra la arquitectura de alto nivel del sistema. El principio de diseño central es que el frontend nunca se comunica directamente con ningún servicio externo: toda petición pasa por el backend de FastAPI, que actúa como única puerta de entrada y punto de orquestación. Esta decisión simplifica el control de versiones de la API, centraliza el manejo de errores y evita exponer los puertos internos de OSRM y OTP al navegador.

El frontend expone la interfaz cartográfica al usuario y delega en el backend el cálculo de rutas, la consulta de horarios GTFS y la inferencia modal. El backend integra cuatro routers diferenciados: `/api/osrm` para rutas viarias por perfil, `/api/otp` para itinerarios multimodales, `/api/gtfs` para información estática de la red de transporte, y `/api/lpmc` para la inferencia de elección modal. Cada router encapsula la lógica de comunicación con su servicio subyacente y expone al frontend una interfaz homogénea independiente de los detalles de cada protocolo.

Arquitectura del Simulador de Movilidad Urbana

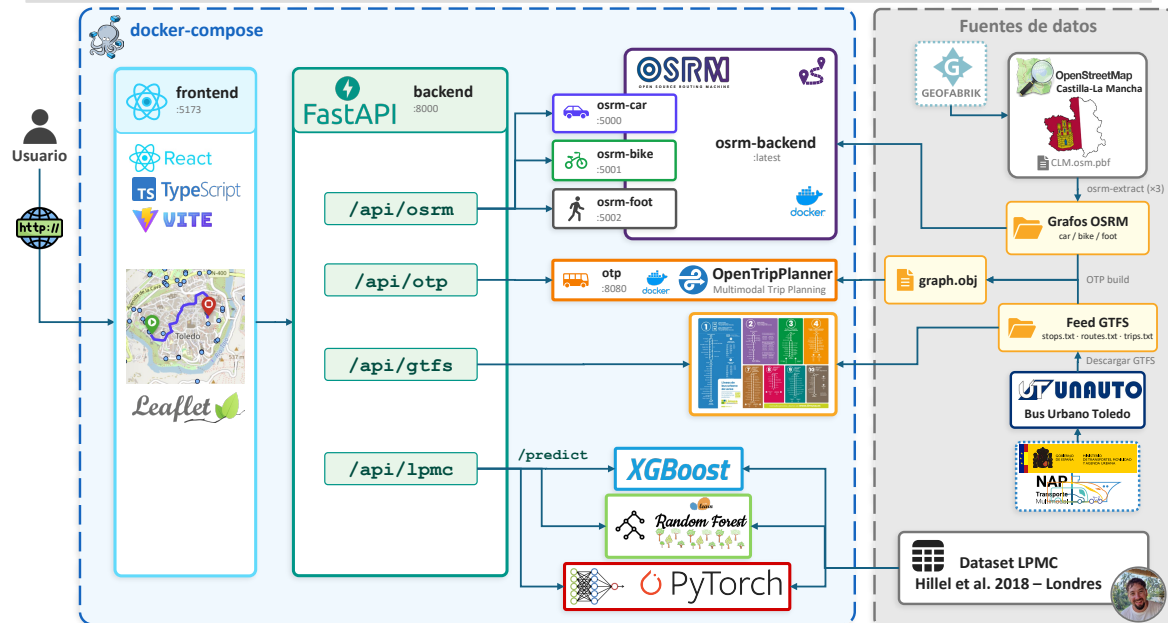


Figura 4.1: Arquitectura general del simulador de movilidad urbana.

4.2. Infraestructura y orquestación

La infraestructura del sistema se define íntegramente en un fichero `docker-compose.yml` en la raíz del repositorio. El despliegue completo se obtiene con un único comando (`docker compose up --build`), lo que elimina dependencias de entorno en la máquina anfitriona y garantiza reproducibilidad.

La Tabla 4.1 resume los servicios definidos, sus puertos y su función en el sistema.

Tabla 4.1: Servicios Docker Compose del simulador.

Servicio	Puerto	Función
frontend	5173	Interfaz web React servida por Vite.
backend	8000	API FastAPI, orquestación y lógica de negocio.
osrm-car	5000	Motor OSRM con perfil de vehículo a motor.
osrm-bike	5001	Motor OSRM con perfil de bicicleta.
osrm-foot	5002	Motor OSRM con perfil peatonal.
otp	8080	OpenTripPlanner 2.x con grafo multimodal de Toledo.

Los servicios OSRM montan los grafos viarios preprocesados desde directorios locales mediante volúmenes Docker. OpenTripPlanner carga el grafo multimodal (`graph.obj`) generado a partir de los datos OSM de Castilla-La Mancha y el feed GTFS urbano de Toledo. Estos artefactos pesados no se versionan en el repositorio Git y deben generarse o descargarse antes del primer despliegue, según se describe en el anexo ??.

La razón por la que OSRM se instancia tres veces en lugar de usar un único servidor multiperfil es que la imagen oficial de OSRM no admite perfiles simultáneos: cada proceso solo puede servir el grafo con el que fue preprocesado. Esta limitación fue el detonante del cambio de la solución inicial basada en el servidor de demostración público, que únicamente ofrecía perfil de conducción, a la infraestructura local multiperfil descrita aquí.

4.3. Backend: capa de orquestación

El backend está implementado con FastAPI sobre Python y se organiza en cuatro routers, cada uno responsable de un dominio funcional. La separación en routers permite desarrollar, probar y documentar cada integración de forma independiente. La Tabla 4.2 recoge los endpoints principales expuestos por cada router.

Tabla 4.2: Endpoints principales de la API REST del backend.

Método	Ruta	Descripción
POST	/api/osrm/routes	Rutas viarias multimodales (OSRM).
POST	/api/otp/routes	Itinerario de transporte público (OTP).
GET	/api/gtfs/stops	Listado de paradas con filtro bbox.
GET	/api/gtfs/routes	Listado de líneas de la red.
GET	/api/gtfs/routes/{id}	Detalle de ruta: paradas y trazado.
GET	/api/gtfs/routes/{id}/schedule	Horarios de una línea por fecha.
POST	/api/lpmc/predict	Inferencia de elección modal con el modelo activo.
POST	/api/lpmc/compare	Inferencia simultánea con XGBoost, RF y DNN.
POST	/api/lpmc/debug-features	Vector de características antes/después del escalado.
GET	/health	Comprobación de disponibilidad.

El router `/api/osrm` recibe un par origen-destino con la lista de perfiles solicitados, consulta en paralelo las instancias OSRM correspondientes y devuelve al frontend las rutas con

distancia, duración y geometría decodificada para su representación cartográfica.

El router `/api/otp` consulta el planificador multimodal y gestiona la paginación de itinerarios alternativos. La respuesta incluye el desglose por tramos (*legs*), con tipo de modo, geometría, parada de inicio y fin, y agencia operadora cuando el tramo es de transporte público. Los itinerarios se ordenan por duración y se selecciona automáticamente el primero que incluya al menos un tramo de transporte público; si ninguno lo incluye, se devuelve el de menor duración.

El router `/api/gtfs` expone información estática de la red de transporte: listado de paradas con sus líneas asociadas, listado de rutas, detalle de ruta con paradas ordenadas y trazado geométrico, y resumen de horarios por fecha. Estos datos se leen directamente del feed GTFS descomprimido en el backend, sin depender de OpenTripPlanner, lo que permite consultar la red aunque el grafo OTP no esté disponible.

El router `/api/lpmc` recibe las coordenadas de origen y destino junto con el perfil sociodemográfico del viajero, construye el vector de variables de entrada al modelo consultando OSRM y OTP internamente, convierte las duraciones a horas para coincidir con las unidades del dataset de entrenamiento, aplica el escalado parcial y ejecuta la inferencia. El endpoint `/predict` usa el modelo activo; `/compare` ejecuta los tres modelos (XGBoost, Random Forest, DNN) sobre el mismo viaje y devuelve sus probabilidades comparadas; `/debug-features` expone el vector de características antes y después del escalado para validación del pipeline.

4.4. Frontend: interfaz cartográfica

El frontend está desarrollado con React, Vite y TypeScript. La aplicación se estructura en dos componentes principales: `App`, que concentra el estado global y la lógica de negocio del cliente, y `MapView`, que encapsula la representación cartográfica mediante React-Leaflet.

`App` gestiona los puntos de origen y destino, el modo de transporte activo, los resultados de rutas, la capa GTFS y el perfil de usuario para la inferencia. Las peticiones al backend se realizan con TanStack Query (`useQuery` y `useMutation`), que proporciona caché, gestión de estados de carga y reintento automático. El estado local de React es suficiente para el alcance del prototipo: no existe flujo de datos compartido entre componentes independientes que justifique una biblioteca adicional de gestión de estado.

`MapView` recibe el estado relevante como *props* y se encarga exclusivamente de la renderización cartográfica: marcadores de origen y destino, polilíneas de ruta coloreadas según el modo activo, paradas GTFS con popups interactivos y segmentos OTP diferenciados vi-

sualmente entre tramos a pie y tramos en vehículo. Los segmentos de transporte público solo se muestran cuando el modo activo es *transit*, evitando solapamientos visuales con las rutas viarias. La capa de fondo es configurable entre cuatro basemaps: carto-light (analítico, sin distracciones visuales), OpenStreetMap estándar (cartografía en color), OpenTopoMap (relieve con curvas de nivel) y satélite Esri (imágenes aéreas).

4.5. Pipeline de inferencia modal

La inferencia de elección modal es la operación más compleja del sistema porque requiere coordinar tres servicios externos de forma concurrente para construir el vector de entrada al modelo. La Figura 4.2 ilustra el flujo completo.

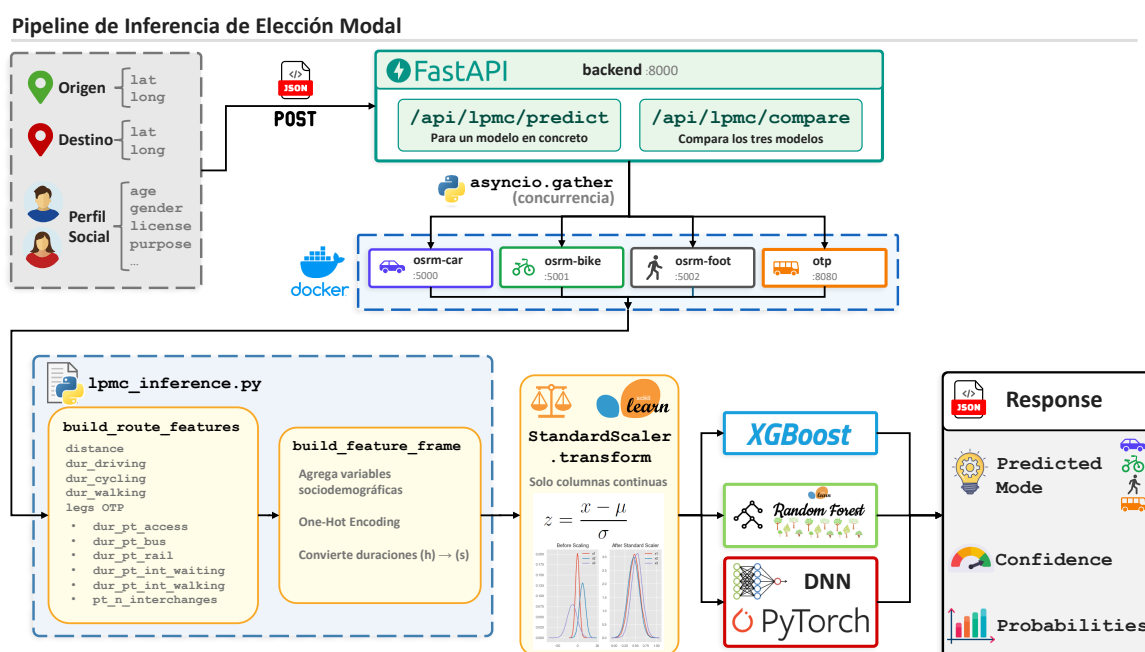


Figura 4.2: Pipeline de inferencia de elección modal.

Al recibir una petición POST `/api/lpmc/predict`, el backend lanza en paralelo cuatro consultas asíncronas mediante `asyncio.gather`: una a cada una de las tres instancias OSRM (conducción, bicicleta, a pie) y una a OpenTripPlanner. Con los resultados se construye el vector de características de ruta, se añaden las variables sociodemográficas del perfil recibido en la petición y se ensambla el vector completo de entrada al modelo.

Las variables de entrada al modelo proceden de tres fuentes diferenciadas: los tiempos y

distancias por perfil de transporte calculados por OSRM, el desglose temporal del itinerario de transporte público proporcionado por OTP, y los datos sociodemográficos y contextuales que el usuario introduce en el formulario del panel lateral. El capítulo 5 detalla la construcción del vector de características y las transformaciones aplicadas.

Las variables continuas que presentaban distribución asimétrica en el conjunto de entrenamiento se normalizan con un escalado parcial aplicado solo a las columnas correspondientes, preservando las variables binarias y de conteo sin transformar. El modelo devuelve un vector de cuatro probabilidades correspondientes a los modos *walk*, *cycle*, *pt* y *drive*; el modo predicho es el de mayor probabilidad.

El modelo activo para `/predict` se controla mediante la variable de entorno `LPMC_MODEL_VARIANT` (`xgb` por defecto). Ninguno de los tres modelos incluye `household_id` como variable de entrada, ya que ese identificador de hogar no está disponible en tiempo de ejecución; solo se usó durante el entrenamiento como criterio de agrupación en la validación cruzada.

izar: el
a usar
ser un
tro de la
POST /a-
/predict (ej.
" : "xgb"),
boost co-
r por de-
a variable
rno es un
de desarro-
no debería
produc-

5. Implementación y resultados

Este capítulo detalla el proceso de construcción del simulador: las decisiones técnicas concretas tomadas en cada componente, los problemas que surgieron durante el desarrollo así como la solución adoptada en cada caso, y los resultados del entrenamiento y evaluación de los modelos de elección modal.

El sistema implementado está disponible como código abierto en <https://github.com/ivanuclm/tfm>, lo que permite su reproducción, extensión y reutilización en futuros trabajos de investigación, desarrollo o divulgación sobre movilidad urbana. El procedimiento de despliegue para reproducir el entorno local se detalla en el anexo ??.

5.1. Infraestructura de enrutado viario: OSRM

5.1.1. Origen y características del extracto OSM

Open Source Routing Machine (OSRM) trabaja sobre un extracto de OpenStreetMap (OSM), un fichero binario con extensión `.osm.pbf` que contiene la geometría de calles, carreteras y caminos de un territorio junto con sus atributos de velocidad, sentido y accesibilidad por modo. Geofabrik [Geofabrik GmbH, 2026] distribuye estos extractos por jerarquía geográfica, desde continentes hasta comunidades autónomas.

Se utilizó el extracto de Castilla-La Mancha al ser la región en la que se enmarca el proyecto, escogiendo Toledo como caso de estudio definitivo, tal como se detalla en la sección 5.2. OpenTripPlanner reutiliza el mismo extracto para la red viaria peatonal, por lo que un único fichero sirve a dos servicios del sistema. La Tabla 5.1 resume los extractos de OSM evaluados durante el desarrollo.

Tabla 5.1: Comparativa de extractos OSM disponibles en Geofabrik.

Extracto	Tamaño (.osm.pbf)	Uso en el proyecto
Castilla-La Mancha	~98 MB	Producción (OSRM y OTP)
Comunidad de Madrid	~79 MB	Pruebas con feed EMT Madrid
España	~1,3 GB	Evaluated, descartado por tamaño
Europa	~32,0 GB	Descartado por tamaño

5.1.2. Evolución del despliegue

El despliegue de OSRM pasó por tres iteraciones antes de conseguir la configuración definitiva.

Fase 1: servidor de demostración oficial. La primera implementación utilizó el servidor de demostración público del proyecto OSRM (`router.project-osrm.org`) [?]. Este servidor ofrece acceso gratuito a la API de enrutado sin necesidad de infraestructura local y está pensado para pruebas rápidas de integración. El problema es que solo procesa el perfil de conducción independientemente del indicado en la URL [danpat, 2017]; cambiar el perfil en la petición no tiene efecto, lo que hacía inviable la comparación multimodal que el simulador requiere.

Fase 2: instancias públicas FOSSGIS. Las instancias públicas mantenidas por FOSSGIS [FOSSGIS e.V., 2026] sí devuelven rutas diferenciadas por perfil mediante endpoints independientes (`/routed-car`, `/routed-bike`, `/routed-foot`), lo que permitió confirmar el funcionamiento multimodal y ajustar la integración del backend. El inconveniente es que estos servicios aplican limitaciones de uso (*rate limiting*), presentan latencia variable y no permiten fijar la versión de los datos, lo que compromete la reproducibilidad.

Fase 3: instancia local con Docker. La solución definitiva fue desplegar OSRM localmente mediante la imagen oficial `osrm/osrm-backend` [?], con un contenedor independiente por modo de transporte (conducción, bicicleta y a pie; el transporte público en autobús es responsabilidad de OpenTripPlanner). Esta arquitectura elimina la dependencia de servicios externos en tiempo de ejecución y garantiza reproducibilidad, control de versión y ausencia de límites de cuota. El procedimiento completo de despliegue se recoge en el anexo ??.

Ver anexo de
que: docker
e up (des-
en un clic)
nalmente,
os OSRM
es paso a

5.1.3. Pipeline de preprocesado por perfil

Para que OSRM pueda calcular rutas, el extracto `.osm.pbf` debe transformarse en una estructura de datos interna optimizada para ejecutar consultas y obtener el camino mínimo entre dos puntos del mapa. Este preprocesado se ejecuta una sola vez por perfil y solo hay que repetirlo si se actualiza el extracto de OSM.

Cada perfil define las reglas de movilidad del modo correspondiente: qué tipos de vía son accesibles, a qué velocidad y con qué restricciones de giro. Los tres archivos de perfil estándar (`car.lua`, `bicycle.lua` y `foot.lua`) están incluidos en la imagen oficial de Docker `osrm/osrm-backend` en `/opt/`, por lo que no es necesario descargarlos ni configurarlos externamente. Todas las etapas del preprocesado se ejecutan con esa misma imagen, lo que garantiza que el preprocesado sea reproducible en cualquier entorno de ejecución.

1. **Extracción** (`osrm-extract`): lee el `.osm.pbf` y construye el grafo viario interno aplicando las reglas del perfil Lua indicado: qué vías son transitables, a qué velocidad y con qué restricciones. El resultado es el fichero `.osrm` junto con varios auxiliares de índice.
2. **Particionado** (`osrm-partition`): divide el grafo en celdas jerárquicas para el algoritmo Multi-Level Dijkstra (MLD) [Geisberger et al., 2008]. Esta partición es la que permite calcular rutas de larga distancia en milisegundos, al limitar la búsqueda a fronteras entre celdas en lugar de explorar el grafo completo.
3. **Personalización** (`osrm-customize`): asigna los pesos finales de cada arista sobre la estructura de celdas anterior, produciendo el grafo listo para servir peticiones de ruta.

Las tres etapas generan los ficheros `clm.osrm` y sus auxiliares, almacenados en directorios separados por perfil (`osrm-clm/car/`, `osrm-clm/bike/`, `osrm-clm/foot/`). Estos archivos no se versionan en el repositorio por su tamaño, pero el procedimiento completo para su generación se recoge en el anexo ??.

El preprocesado completo de los tres perfiles sobre el extracto de Castilla-La Mancha tarda aproximadamente 97 segundos en la máquina de desarrollo. Los artefactos generados ocupan en total unos 2,1 GB: el perfil de conducción produce 291 MB porque solo incorpora la red viaria accesible al tráfico rodado, mientras que los perfiles de bicicleta y peatonal alcanzan los 911 MB cada uno al incluir además senderos, caminos rurales y zonas de acceso restringido al tráfico.

Decidir si versionar con Git LFS; actualmente excluidos del repositorio por tamaño

Completar anexo de despliegue con los comandos `osrm-extract`, `osrm-partition`, `osrm-customize` por perfil

Pipeline de preprocesado OSRM por perfil de transporte

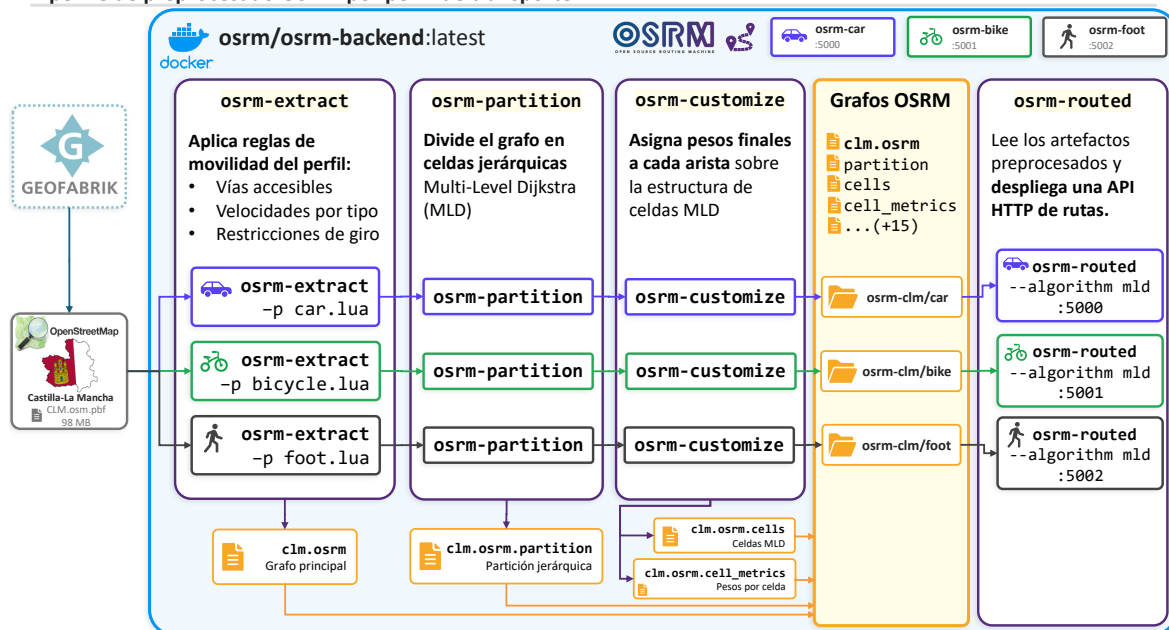


Figura 5.1: Pipeline de preprocesado OSRM por perfil de transporte.

5.1.4. Despliegue y verificación

Cada uno de los tres contenedores OSRM ejecuta el servidor de enrutado sobre su grafo preprocesado con el algoritmo MLD (Figura 5.1):

```
osrm-routed --algorithm mld /data/clm.osrm
```

Los tres contenedores comparten imagen y comando de arranque. Cada uno monta el directorio de artefactos de su perfil (`osrm-clm/car`, `osrm-clm/bike`, `osrm-clm/foot`). Dentro de la red interna de Docker Compose, el proceso `osrm-routed` escucha siempre en el puerto 5000 en todos los contenedores, y el backend los referencia por nombre de servicio (`osrm-car`, `osrm-bike`, `osrm-foot`) en ese mismo puerto. Para verificar las instancias directamente desde el terminal del anfitrión, cada contenedor expone un puerto distinto hacia el exterior: 5000, 5001 y 5002, respectivamente para los perfiles de conducción, bicicleta y a pie.

Una respuesta válida devuelve `"code": "Ok"` junto con la distancia en metros, la duración en segundos y la geometría en formato polilínea [Google LLC,], una cadena de texto con la secuencia de coordenadas de la ruta, que el backend decodifica antes de enviarla al frontend. Si el grafo está incompleto o el perfil es incorrecto, OSRM devuelve `"code": "NoRoute"` o una ruta degradada, detectable al comparar visualmente los tres trazados en la interfaz.

```
# osrm-car (conduccion, localhost:5000)
curl "http://localhost:5000/route/v1/driving
      /-4.0356,39.8750;-4.0023,39.8602?overview=full"
# osrm-bike (bicicleta, localhost:5001)
curl "http://localhost:5001/route/v1/cycling
      /-4.0356,39.8750;-4.0023,39.8602?overview=full"
# osrm-foot (peatonal, localhost:5002)
curl "http://localhost:5002/route/v1/foot
      /-4.0356,39.8750;-4.0023,39.8602?overview=full"
```

Código 5.1: Verificación de las tres instancias OSRM desde el terminal del anfitrión. El parámetro `overview=full` fuerza la devolución de la geometría completa de la ruta; sin él, OSRM devuelve una versión simplificada con menos puntos.

La Figura 5.2 muestra las tres rutas calculadas simultáneamente para el mismo par origen-destino: en azul, la ruta de conducción sigue las calles principales; en verde, la ruta de bicicleta prioriza carriles bici y calles tranquilas; en gris con línea discontinua, la ruta peatonal puede acceder a zonas restringidas al tráfico rodado. Los tres trazados ponen de manifiesto las distintas restricciones de cada perfil sobre la red viaria de Toledo.

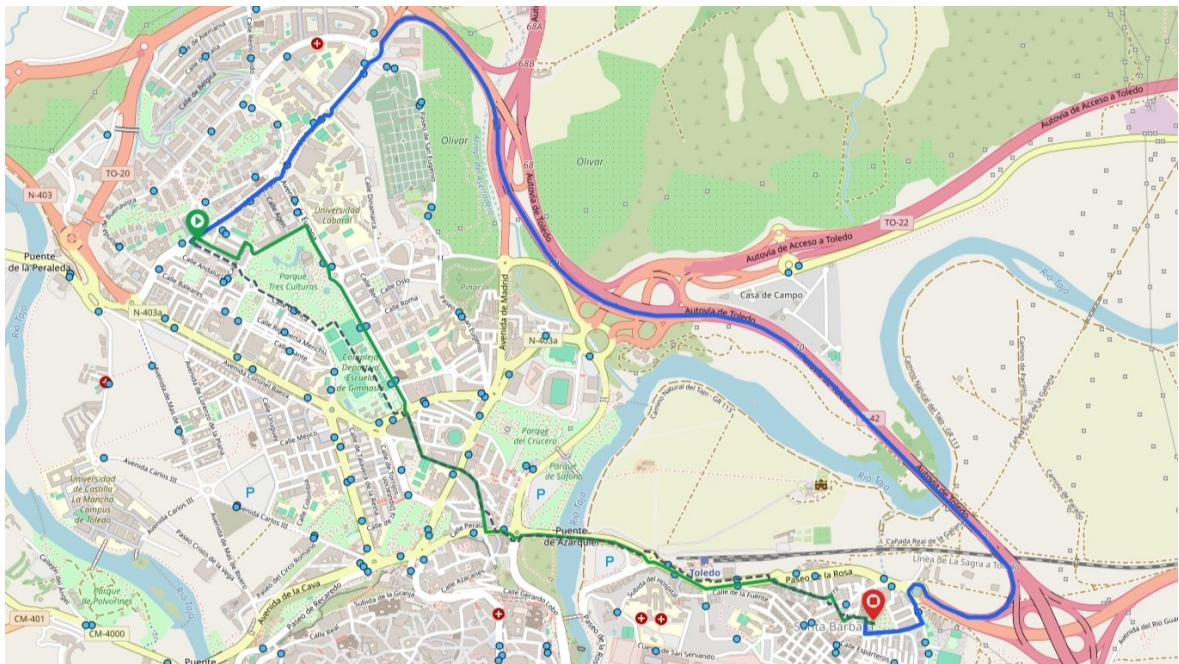


Figura 5.2: Rutas OSRM para los tres perfiles de transporte sobre Toledo.

5.2. Planificación multimodal: OpenTripPlanner

5.2.1. Fuente de datos GTFS y proceso de selección

OpenTripPlanner requiere dos fuentes de datos: un extracto viario OSM para modelar los tramos de acceso a pie, y un feed GTFS con el servicio programado del operador de transporte público para los tramos en vehículo. Al enmarcarse el proyecto en Castilla-La Mancha, la búsqueda de feed GTFS de bus urbano se orientó a la región desde el principio, con la ventaja de que el extracto OSM ya estaba disponible por su uso en OSRM (sección 5.1.1). La fuente empleada fue el Punto de Acceso Nacional de Transporte Multimodal (NAP), gestionado por el Ministerio de Transportes y Movilidad Sostenible, que centraliza feeds GTFS de operadores de toda España [NAP Transporte Multimodal, 2026a], con amplia cobertura de servicios interurbanos pero escasa representación del transporte urbano municipal.

Las opciones disponibles en Castilla-La Mancha resultaron ser muy escasas. En Albacete solo figuraban servicios de operadores interurbanos como Alsa, sin datos de red urbana en el NAP. Guadalajara contaba con feeds del Consorcio de Transportes de Madrid (cercanías y autobuses interurbanos de la Comunidad de Madrid), pero sin red municipal propia. Cuenca tampoco disponía de feed urbano. La única opción urbana era Ciudad Real, cuyo feed, operado por AISA, incluía también Seseña, con paradas repartidas entre dos localidades separadas por más de 170 km; además, en ese momento no incluía geometría de rutas, requisito imprescindible para la representación cartográfica del servicio.

Ante la ausencia de un feed GTFS de Castilla-La Mancha válido, se evaluaron feeds de otras ciudades como candidatos alternativos para el proyecto. La EMT de Madrid [EMT Madrid, 2026], con 4.914 paradas, 236 rutas y 87.137 viajes programados, sirvió de banco de pruebas para validar la integración de OTP con datos reales y desarrollar los endpoints de la capa estática del backend. El volumen del feed reveló además un problema de paginación: la API, que exige un límite explícito, devolvía solo las primeras 500 paradas por defecto, dejando la red visualmente incompleta. El límite se ajustó a 5.000, suficiente para Madrid y los feeds posteriores.

El feed GTFS elegido finalmente fue el de los autobuses urbanos de Toledo [?]. Con 268 paradas, 56 rutas y 46.597 viajes que cubren aproximadamente un mes de servicio programado, tiene un tamaño ajustado al alcance del simulador, cubre el casco urbano, el polígono industrial y las urbanizaciones periféricas, e incluye geometría de rutas para su representación cartográfica. Como el feed no incluye tarifas, el coste del billete se introduce como parámetro en el panel lateral de la interfaz (de forma que pueda ser fijado por el analista en la simulación), junto al resto de variables del perfil de viajero.

5.2.2. Vigencia del feed y decisión de fecha fija

Los feeds GTFS del NAP tienen vigencia limitada; para el operador de Toledo, los ficheros publicados cubren aproximadamente tres meses de servicio programado (90 días en los feeds descargados durante el desarrollo). Esto generaba un problema operativo directo. Si la fecha de consulta caía fuera del rango de validez del feed cargado en OTP, el planificador no devolvía itinerarios de transporte público y el simulador producía resultados inservibles para la inferencia modal.

Al reanudar el desarrollo en enero de 2026, la fecha del sistema había superado el rango del feed en uso, por lo que se adoptaron dos medidas: fijar en el backend una fecha y hora estáticas (`date=2025-12-01`, `time=12:00`) siempre dentro del rango del feed descargado, y versionar una copia estable del feed en el directorio de datos del repositorio. Así, todos los experimentos y ejecuciones del simulador trabajan sobre el mismo escenario de oferta de transporte. Durante el desarrollo se descargaron varias versiones del feed; en las primeras el operador figuraba como «Autobuses Toledo» y en posteriores había pasado a denominarse UNAUTO S.L. (Grupo Ruiz), aunque el formato de los ficheros GTFS permaneció idéntico en todos los casos.

Se fijó la hora de consulta a las 12:00 por situarse en la franja de mayor frecuencia de servicio y evitar el horario nocturno, donde la oferta es reducida o inexistente.

5.2.3. Construcción del grafo multimodal

El grafo se construye con un único comando Docker ejecutado sobre el directorio `otp-toledo/`, que debe contener el extracto OSM (`clm.osm.pbf`) y el feed GTFS (`GTFS_Urbano_Toledo.zip`):

```
docker run --rm -v "otp-toledo:/var/opentripplanner"
  opentripplanner/opentripplanner:2.5.0 --build --save
```

OTP lee ambos ficheros y serializa el resultado en `graph.obj`: la red viaria OSM modela los tramos de acceso a pie a las paradas, y el feed GTFS aporta el servicio programado con horarios, paradas y geometría de líneas. El proceso tarda aproximadamente dos minutos en la máquina de desarrollo. Solo es necesario repetirlo cuando se actualiza el feed; el extracto OSM puede reutilizarse entre versiones.

Al igual que los grafos OSRM, el `graph.obj` no se versiona en el repositorio por su tamaño y debe generarse antes del primer despliegue. Una vez disponible, OTP se lanza en modo de explotación (`--load --serve`) y queda accesible en el puerto 8080, exponiendo la API REST de planificación [?].

Completar anexo de despliegue con construcción del grafo OTP: comando `--build --save` y requisitos previos (OSM + GTFS)

5.2.4. Integración de itinerarios en el backend

El router `/api/otp` del backend consulta OTP mediante peticiones HTTP a `/otp/routers/default/plan` y deserializa la respuesta para extraer los tramos (*legs*) del itinerario seleccionado. OTP devuelve hasta cinco itinerarios alternativos. El criterio de selección automática es el siguiente: se elige el primero que incluya al menos un tramo de transporte público (modo distinto de WALK); si ninguno cumple esta condición, se devuelve el de menor duración total. El frontend permite paginar entre las alternativas mediante los botones «Anterior» y «Siguiente», que transmiten el índice del itinerario deseado al backend, garantizando que el vector de características para la inferencia se calcula siempre sobre el mismo itinerario que se visualiza.

Para cada tramo del itinerario seleccionado se devuelve: modo de transporte, distancia, duración, geometría decodificada, parada de inicio y fin, nombre de línea, agencia y horarios de salida y llegada en formato HH:MM. La geometría, codificada en formato polyline, se decodifica en el frontend y se concatena para componer la traza completa del itinerario.

Cuando la distancia entre origen y destino es muy corta, o cuando no existe servicio de autobús entre los dos puntos en el horario fijado, OTP puede devolver como «mejor itinerario» una ruta completamente a pie, compuesta por un único tramo WALK, equivalente en tiempo y trayecto a la ruta que ya proporciona OSRM con el perfil peatonal. En ese caso, si se construyese el vector de características de transporte público con los valores devueltos (tiempo de acceso cero, tiempo en bus cero, transbordos cero), el modelo podría asignar una probabilidad elevada al modo *pt* en trayectos sin servicio real. La solución adoptada, descrita en la sección 5.4.4, consiste en detectar esta situación y sobrescribir los atributos de transporte público del vector de características con valores de penalización que llevan al modelo a descartar ese modo.

La Figura 5.3 ilustra un itinerario multimodal típico en la interfaz del simulador: un tramo a pie inicial (línea discontinua gris) conecta el origen con la parada de embarque, un tramo en autobús (línea continua de color) cubre el recorrido principal, y un tramo a pie final lleva hasta el destino. En el panel lateral se desglosa cada tramo con su modo de transporte, duración en minutos, paradas de inicio y fin, nombre de línea y horario de salida.

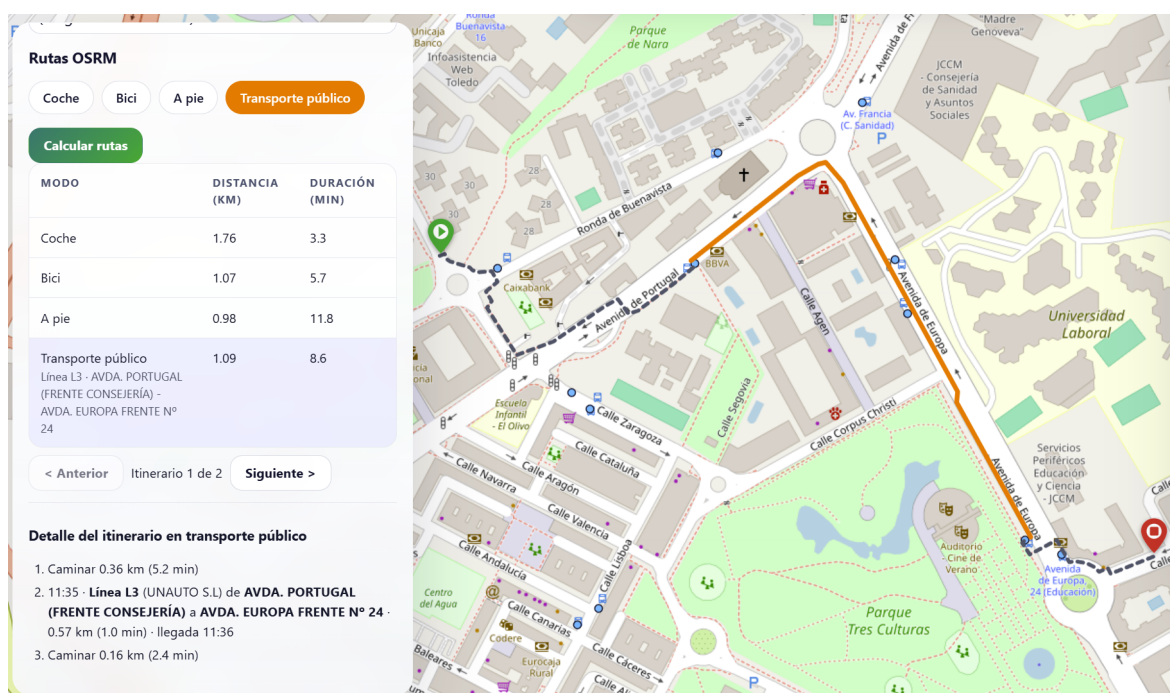


Figura 5.3: Itinerario multimodal OTP con desglose de tramos en la interfaz.

5.3. Integración del feed GTFS

5.3.1. Arquitectura de la capa estática

El router `/api/gtfs` expone la información estática de la red de autobús directamente a partir de los ficheros del feed GTFS, sin depender de OpenTripPlanner. Esta separación tiene una ventaja operativa concreta: la red puede consultarse aunque OTP no esté disponible, lo que resultó útil durante el desarrollo para depurar la visualización de paradas y líneas sin necesidad de levantar el planificador ni construir el grafo.

El feed GTFS es una colección de ficheros CSV empaquetados en un archivo ZIP [MobilityData, 2026b]. En el arranque del servicio, el backend descomprime y carga en memoria con pandas los seis ficheros necesarios: `stops.txt` (paradas con coordenadas y nombre), `routes.txt` (líneas de la red), `trips.txt` (servicios programados por línea), `stop_times.txt` (horarios de paso por cada parada), `shapes.txt` (geometría de los trazados) y `calendar_dates.txt` (calendario de excepciones de servicio). Los DataFrames resultantes permanecen en memoria durante toda la vida del proceso.

Responder a preguntas como “qué líneas pasan por esta parada” o “cuál es el ho-

rario de una ruta para una fecha concreta” requiere cruzar varios de estos ficheros. Para obtener las líneas asociadas a una parada se encadenan tres joins: `stops.txt` → `stop_times.txt` → `trips.txt` → `routes.txt`. Para los horarios por fecha se añade el filtro de `calendar_dates.txt`, que indica los días en que cada servicio está activo. Mantener todos los ficheros en memoria evita releer los CSV en cada petición y elimina la necesidad de añadir una base de datos para datos que no cambian en tiempo de ejecución.

5.3.2. Endpoints disponibles

La capa GTFS expone cuatro endpoints que cubren las operaciones necesarias para la visualización cartográfica y la consulta de la red:

- **Listado de paradas** (GET `/api/gtfs/stops`): devuelve todas las paradas del feed con identificador, nombre, coordenadas y líneas asociadas. Acepta un parámetro `limit` (5.000 por defecto, suficiente para el feed de Toledo con 268 paradas) y un filtro de bounding box opcional para restringir el resultado al área visible en el mapa.
- **Listado de rutas** (GET `/api/gtfs/routes`): catálogo completo de líneas con identificador, nombre corto, nombre largo y atributos de color cuando están disponibles en el feed.
- **Detalle de ruta** (GET `/api/gtfs/routes/{route_id}`): dado el identificador de una línea, devuelve la secuencia ordenada de paradas y la geometría del *shape* asociado para su representación cartográfica.
- **Horarios por fecha** (GET `/api/gtfs/routes/{route_id}/schedule`): dado un identificador de ruta y una fecha (`?date=YYYY-MM-DD`), devuelve los servicios programados agrupados por dirección, con primera y última salida y listado completo de horarios. Al igual que con OTP (sección 5.2.2), la interfaz usa una fecha fija dentro del rango de validez del feed en lugar de la fecha del sistema, por la misma razón: el feed cubre un periodo de aproximadamente tres meses y cualquier fecha posterior devuelve cero servicios.

El feed de Toledo contiene 56 entradas en `routes.txt`, pero solo 25 líneas físicas distintas: el operador crea un `route_id` independiente por cada combinación de dirección y variante de recorrido de una misma línea. La interfaz agrupa las entradas por nombre corto (`route_short_name`) y muestra únicamente las 25 líneas únicas, evitando duplicados en el catálogo visual.

5.3.3. Coloración de líneas y agrupación por sentido

El feed GTFS de Toledo incluye los campos `route_color` y `route_text_color` en `routes.txt` para todas sus entradas. La función `routeColor()` los lee directamente y construye el código hexadecimal correspondiente, garantizando coherencia visual con la señalética del operador. Para feeds que no incluyan estos campos, la misma función genera un color de respaldo mediante un hash polinómico sobre el `route_short_name` (o el identificador de ruta si no existe nombre corto). El hash proyecta la cadena a un entero cuyo módulo 360 determina el tono; la saturación y la luminosidad se fijan a 70 % y 35 % respectivamente, valores que aseguran contraste suficiente para texto blanco superpuesto independientemente del tono generado. El mismo valor de entrada siempre produce el mismo color, por lo que la asignación es estable entre renders y sesiones. El Código 5.1 muestra las dos funciones.

Listing 5.1: Color de línea: feed GTFS como fuente primaria y hash determinista como respaldo.

```
function hslForKey(key: string): string {
  let h = 0;
  for (let i = 0; i < key.length; i++) h = (h * 31 + key.charCodeAt(i)) | 0;
  return `hsl(${Math.abs(h) % 360}, 70%, 35%)`;
}

function routeColor(r: { color?: string | null; short_name?: string | null; id: string }): string {
  if (r.color) return `#${r.color}`;
  return hslForKey(r.short_name || r.id);
}
```

El feed de Toledo contiene 56 entradas en `routes.txt` pero solo 25 líneas físicas distintas, ya que el operador registra cada sentido de circulación como un `route_id` independiente con el mismo `route_short_name`. La interfaz deduplica las entradas por nombre corto para el catálogo visual y agrupa en un diccionario (`routeSiblingsByShortName`) los identificadores de ruta que comparten nombre. Cuando el usuario selecciona una línea, el índice de la variante activa se deriva de la posición del `route_id` seleccionado dentro de la lista de rutas hermanas, de modo que un clic en un chip de parada del mapa, que conoce el `route_id` exacto del servicio que pasa por esa parada, activa directamente el sentido correcto sin lógica adicional.

La Figura 5.4 muestra el panel Red GTFS. El catálogo se organiza en un acordeón donde cada fila representa una línea física con su badge coloreado y nombre largo. Al expandir una

entrada con varios sentidos, aparece una sublista de rutas hermanas con el identificador activo resaltado mediante un borde lateral del color de la línea. El panel muestra además un diagrama vertical de paradas inspirado en los paneles de andén: línea vertical del color de la línea con terminales como puntos rellenos, paradas intermedias como aros huecos y la parada de referencia resaltada en azul. Al pulsar sobre el nombre de una parada, el mapa ejecuta un desplazamiento animado (`flyTo`) hasta esa posición. Una tabla de salidas agrupada por hora completa la información de la ruta seleccionada.

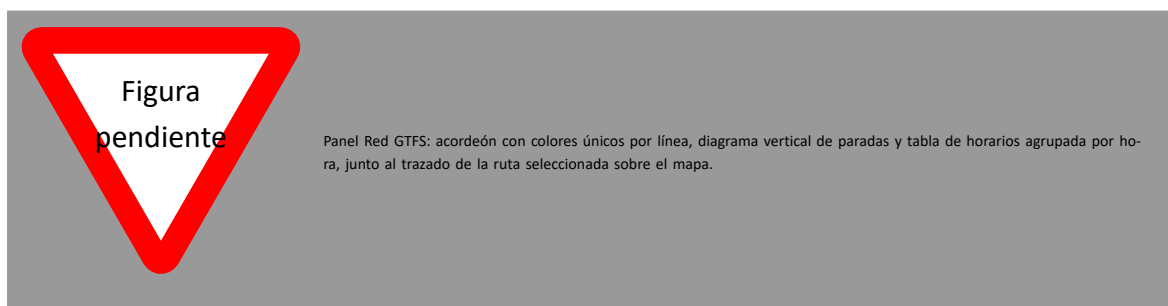


Figura 5.4: Panel GTFS con acordeón de líneas, diagrama de paradas y horarios.

5.4. Implementación del backend FastAPI

5.4.1. Estructura modular

El backend está implementado con FastAPI sobre Python y organizado en módulos por responsabilidad. Un directorio `api/` contiene los cuatro routers (`routes_osrm.py`, `routes_otp.py`, `routes_gtfs.py`, `routes_lpmc.py`); un directorio `services/` contiene la lógica de negocio desacoplada de la capa HTTP (`osrm_client.py`, `lpmc_inference.py`); y el módulo raíz `main.py` monta los routers y configura CORS. Los endpoints expuestos por cada router se recogen en la Tabla 4.2 del capítulo 4.

La separación en módulos permite desarrollar, probar y documentar cada router de forma independiente. FastAPI genera documentación OpenAPI automática en `/docs` que refleja el estado actual de los endpoints en tiempo real, lo que facilitó la verificación del contrato de cada integración durante el desarrollo sin necesidad de un cliente HTTP externo.

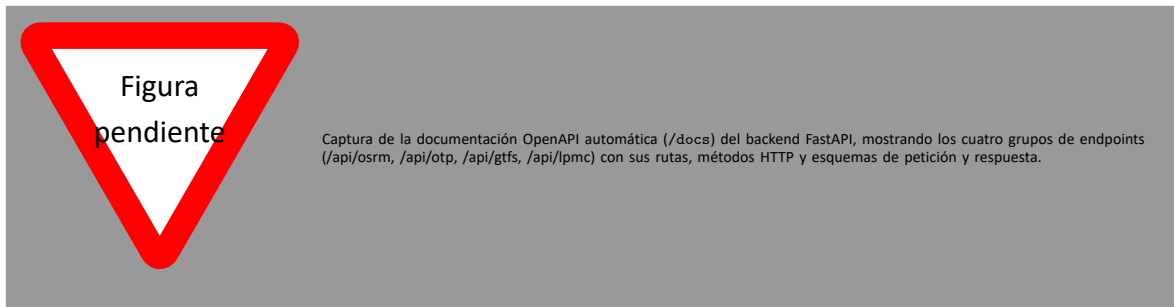


Figura 5.5: Documentación OpenAPI generada automáticamente por FastAPI.

5.4.2. Router OSRM: consultas multiperfil

El router `/api/osrm/routes` acepta un par origen-destino y una lista de perfiles solicitados, y lanza las consultas a los tres contenedores OSRM en paralelo mediante `asyncio.gather`. La paralelización reduce la latencia total al tiempo de la consulta más lenta en lugar de la suma de las tres, lo que resulta relevante cuando el usuario solicita los tres perfiles simultáneamente. Para cada perfil, el backend recibe la distancia en metros, la duración en segundos y la geometría en formato polyline, decodifica esta última en una lista de coordenadas `[lat, lon]` lista para Leaflet, y convierte las duraciones de segundos a horas antes de construir el vector de características, ya que el dataset LPMC almacena todas las duraciones en esa unidad.

5.4.3. Router OTP: itinerarios multimodales

El router `/api/otp/routes` consulta el endpoint de planificación de OTP, gestiona la selección de itinerario (automática o por índice explícito recibido desde el frontend) y normaliza la respuesta. Para cada tramo del itinerario seleccionado se devuelve: modo de transporte, distancia, duración, geometría decodificada desde polyline, parada de inicio y fin, nombre de línea, agencia y horarios de salida y llegada. La geometría de cada tramo se concatena para formar la traza completa del itinerario, que el frontend divide y renderiza por segmentos según el modo.

5.4.4. Penalización de itinerarios sin transporte público real

Cuando OTP devuelve un itinerario compuesto exclusivamente por un tramo `WALK`, sin ningún tramo de transporte público, las variables de ruta `PT` se sobrescriben con un valor de penalización antes de construir el vector de características. Este comportamiento se iden-

tificó durante las pruebas de integración del módulo de inferencia: para trayectos cortos en los que la opción más rápida era ir a pie, OTP devolvía ese único tramo como itinerario «multimodal», y el vector de variables PT resultante, con valores próximos a cero, podía inducir al modelo a asignar una probabilidad elevada al modo *pt* en ausencia de servicio real.

La solución, acordada con el tutor, consiste en comprobar si el itinerario contiene algún tramo de modo distinto de WALK. Si no lo contiene, las variables `dur_pt_access`, `dur_pt_bus`, `dur_pt_rail`, `dur_pt_int_waiting`, `dur_pt_int_walking` y `pt_n_interchanges` se asignan con un valor situado en el extremo superior de su distribución en el conjunto de entrenamiento. De este modo el modelo percibe la opción de transporte público como un itinerario con tiempo de viaje muy elevado, desincentivando su selección sin necesidad de modificar la arquitectura de inferencia ni el número de variables de entrada.

5.4.5. Entorno Docker y resolución de rutas

El backend se ejecuta dentro de un contenedor Docker que monta el repositorio completo en `/workspace`. Esta configuración permite que tanto el código del backend como los artefactos del modelo (almacenados en `lpmc/models/`) sean accesibles desde el mismo sistema de ficheros sin necesidad de copiarlos al contenedor durante la construcción de la imagen. La función `_project_root()` del módulo de inferencia resuelve la ruta del modelo mediante `Path(__file__).resolve().parents[4]`, navegando desde el módulo hasta la raíz del repositorio y localizando `lpmc/models/` de forma independiente del punto de montaje concreto. Adicionalmente, el código normaliza los separadores de ruta antes de extraer el nombre de fichero del bundle serializado, lo que garantiza la portabilidad entre el entorno de desarrollo Windows y el contenedor Linux de producción.

5.5. Implementación del frontend

5.5.1. Tecnologías y estructura

El frontend está desarrollado con React 18, Vite y TypeScript. Vite proporciona recarga en caliente instantánea durante el desarrollo y genera el bundle de producción con división de código automática. TypeScript añade comprobación estática de tipos, especialmente útil al manejar las estructuras de datos complejas que devuelven los endpoints del backend: itinerarios OTP con desglose por tramos, respuestas GTFS con múltiples entidades relacionadas

y vectores de probabilidades de los tres modelos. Leaflet se integra a través de React-Leaflet como librería cartográfica.

La aplicación está estructurada en dos componentes principales, `App` y `MapView`, descritos en la sección 4.4. Esta separación, en la que `MapView` recibe todos sus datos como *props* y no mantiene estado propio, simplificó el desarrollo incremental de la capa cartográfica de forma independiente al resto de la lógica.

5.5.2. Interfaz cartográfica

El usuario coloca los puntos de origen y destino haciendo clic en el mapa de forma alternada. Los marcadores se implementaron con iconos personalizados en CSS, en lugar de los predeterminados de Leaflet, para distinguir visualmente el punto de inicio del de llegada. El diseño partió de elementos base de la librería, adaptados mediante estilos propios.

Las rutas viarias calculadas por OSRM se representan como polilíneas coloreadas según el modo: azul para conducción, verde para bicicleta y gris para desplazamiento a pie. Cuando el usuario solicita los tres perfiles simultáneamente, los tres trazados se superponen sobre el mismo mapa, lo que permite comparar visualmente los recorridos y apreciar las diferencias de trayecto entre modos.

La aplicación ofrece cuatro mapas base seleccionables: analítico (CartoDB Light, paleta desaturada que favorece la legibilidad de las rutas superpuestas), color estándar (OpenStreetMap), relieve con curvas de nivel (OpenTopoMap) y vista aérea (Esri World Imagery). La selección persiste mientras el usuario navega entre los distintos modos de transporte.

Las paradas del feed GTFS se muestran como marcadores interactivos; al hacer clic en una parada se abre un popup con el nombre, el código identificador y las líneas que pasan por ella. Al seleccionar una línea desde el popup o desde el desplegable del panel lateral, el mapa dibuja el *shape* de la ruta y el panel muestra la secuencia de paradas y los horarios del día seleccionado. Al hacer clic en una parada del listado lateral, la vista se desplaza hasta ella (`flyTo` a `zoom 17`) y, al concluir la animación, se abre automáticamente el popup correspondiente en el mapa.

5.5.3. Representación de itinerarios multimodales

Los itinerarios de transporte público devueltos por OTP tienen una estructura de tramos heterogénea: un viaje típico en autobús urbano se compone de un tramo a pie hasta la parada de embarque, uno o varios tramos en autobús y un tramo a pie final hasta el destino. Representar este itinerario como una única polilínea homogénea perdería la información

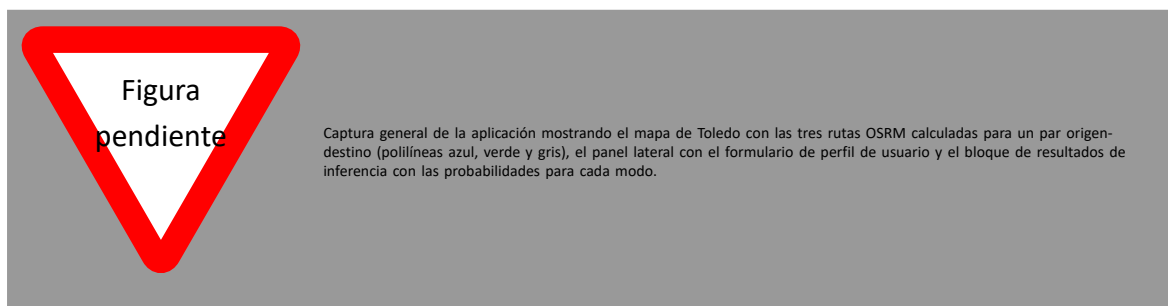


Figura 5.6: Vista general de la interfaz del simulador de movilidad urbana.

sobre qué parte del recorrido se realiza en cada modo.

La solución adoptada consiste en renderizar cada tramo como una polilínea independiente con estilo diferenciado: los tramos a pie se representan con línea discontinua gris y los tramos en autobús con línea continua de color. Las polilíneas del itinerario OTP solo se muestran cuando el modo activo en la interfaz es «Transporte público», evitando el solapamiento visual con las rutas viarias de OSRM cuando el usuario explora otros modos.

Esta distinción requirió adaptar el proceso de renderización para gestionar una lista variable de geometrías con estilos distintos, en lugar del único trazado por consulta que se usa en el caso de OSRM.

5.5.4. Gestión del estado con TanStack Query

Las peticiones al backend se gestionan con TanStack Query ([@tanstack/react-query](https://tanstack.com/react-query)). Los datos GTFS estáticos (paradas, lista de rutas) se cargan con `useQuery` al arrancar la aplicación y se mantienen en caché durante la sesión. Las peticiones de rutas OSRM, itinerarios OTP e inferencia LPMC se declaran como mutaciones (`useMutation`), que se disparan manualmente al pulsar los controles de cálculo en la interfaz.

TanStack Query gestiona de forma automática los estados de carga y error, la caché y el reintento con *backoff* exponencial. Las mutaciones de inferencia LPMC (`/predict`, `/compare`, `/debug-features`) se declaran de forma independiente, lo que permite invocar la predicción con el modelo activo o la comparación de los tres modelos sin afectar al estado del resto de la interfaz.

5.5.5. Perfiles de usuario predefinidos

Para facilitar la exploración del simulador sin necesidad de introducir manualmente todos los parámetros sociodemográficos, se implementaron tres perfiles predefinidos que au-

to completan el formulario del panel lateral:

- **Commuter:** viaje al trabajo (HBW) en martes a las 8:15. Varón de 36 años con carnet de conducir y un coche en el hogar.
- **Estudiante:** viaje a la educación (HBE) en miércoles a las 7:45. Mujer de 21 años sin carnet de conducir ni coche en el hogar, con tarifa de transporte público reducida.
- **Familiar:** viaje de ocio (HBO) en sábado a las 11:30. Mujer de 44 años con carnet de conducir y dos coches en el hogar.

Al activar un perfil se carga el conjunto completo de valores en el formulario. El usuario puede modificar cualquier campo individualmente para explorar la sensibilidad del modelo a cada parámetro. La detección de «perfil activo» se implementa comparando el estado actual del formulario con los valores predefinidos: el indicador visual desaparece en cuanto el usuario modifica algún campo, eliminando la ambigüedad sobre qué conjunto de valores se está usando en la inferencia.

Los tres perfiles representan escenarios típicos de elección modal con comportamientos esperados claramente diferenciados. La ausencia de carnet de conducir en el perfil de estudiante elimina prácticamente la opción *drive* de la predicción, mientras que la disponibilidad de dos vehículos en el perfil familiar, combinada con el contexto de ocio en sábado, tiende a favorecer el modo *drive* frente al transporte público. Estos contrastes son útiles para verificar cualitativamente que el modelo responde de forma coherente a las variables de entrada.

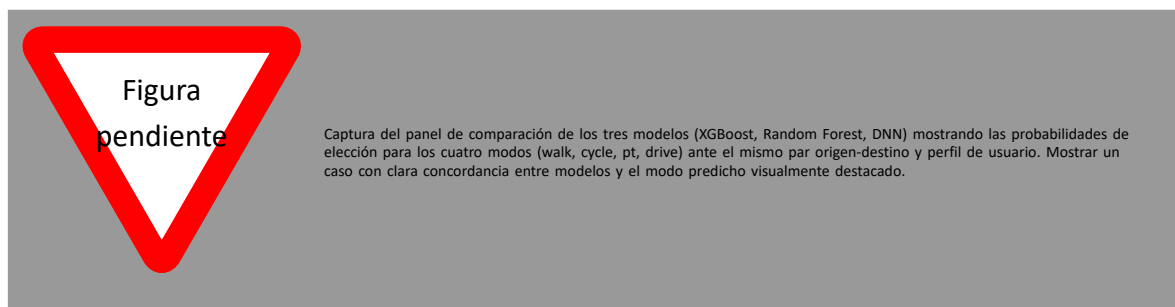


Figura 5.7: Panel de comparación de probabilidades de los tres modelos de elección modal.

5.6. Modelos de elección modal

Esta sección describe el proceso de entrenamiento, validación y despliegue en producción de los tres modelos de elección modal que integra el simulador. La arquitectura del pipeline de inferencia se describe en la sección 4.5; aquí se detalla la implementación concreta: el dataset utilizado, las decisiones de preprocesado, la configuración de los modelos y los resultados obtenidos.

5.6.1. Dataset LPMC y variables de entrada

El dataset London Passenger Mode Choice (LPMC) [Hillel et al., 2018, Hillel, 2019] contiene registros de viajes observados en el área metropolitana de Londres, enriquecidos con variables de nivel de servicio calculadas para cada modo de transporte disponible en cada par origen-destino. Fue construido por Hillel et al. como banco de pruebas para la comparación de modelos de elección discreta y de aprendizaje automático en el dominio del transporte, y es la fuente de la investigación previa del tutor del TFM [Martín-Baos, 2023, José Ángel Martín-Baos, 2023]. La Tabla 5.2 resume sus características principales.

Tabla 5.2: Características generales del dataset LPMC.

Característica	Valor
Registros totales	81.086
Hogares únicos	17.616
Modos de viaje	4 (<i>walk, cycle, pt, drive</i>)
Oleadas de encuesta	3 (<i>survey_year</i> 1, 2, 3)
Años de viaje cubiertos	2012–2015
Variables originales	34

El interés metodológico del dataset para este TFM no reside en el caso geográfico concreto (Londres), sino en que dispone de variables de nivel de servicio equivalentes a las que OSRM y OTP pueden calcular para cualquier par origen-destino, incluye variables sociodemográficas similares a las recogidas por encuestas de movilidad estándar, y tiene un tamaño suficiente para entrenar modelos de aprendizaje automático con validación cruzada robusta.

Las variables de entrada al modelo se dividen en dos grupos. El primero comprende las variables de ruta, derivadas automáticamente de OSRM y OTP para cada par origen-destino

consultado en el simulador. El segundo comprende las variables sociodemográficas y contextuales, introducidas por el usuario a través del panel lateral de la interfaz. La Tabla 5.3 describe ambos grupos con su tipo de dato y su origen.

5.6.2. Preprocesado de datos

5.6.3. Ajuste de hiperparámetros

5.6.4. Entrenamiento y evaluación

Tabla 5.3: Variables de entrada al modelo de elección modal. Las variables de ruta se derivan automáticamente de OSRM y OTP; las sociodemográficas las aporta el usuario a través de la interfaz.

Variable	Tipo / Rango	Descripción
<i>Variables de ruta (derivadas de OSRM y OTP)</i>		
distance	float, m	Distancia en coche (OSRM driving).
dur_driving	float, h	Tiempo de conducción (OSRM driving).
dur_cycling	float, h	Tiempo en bicicleta (OSRM cycling).
dur_walking	float, h	Tiempo a pie hasta destino (OSRM foot).
dur_pt_access	float, h	Tiempo del primer tramo a pie hasta la parada (OTP).
dur_pt_bus	float, h	Tiempo acumulado en autobús (OTP).
dur_pt_rail	float, h	Tiempo acumulado en ferroviario (OTP).
dur_pt_int_waiting	float, h	Tiempo de espera en transbordos (OTP).
dur_pt_int_walking	float, h	Desplazamiento a pie entre transbordos (OTP).
pt_n_interchanges	int, ≥ 0	Número de transbordos (OTP).
<i>Variables sociodemográficas y contextuales (aportadas por el usuario)</i>		
age	int, [16, 100]	Edad del viajero.
female	int, {0, 1}	Sexo (1 = mujer).
driving_license	int, {0, 1}	Posesión de carnet de conducir.
car_ownership	int, [0, 3]	Número de coches en el hogar.
day_of_week	int, [1, 7]	Día de la semana.
start_time_linear	float, [0, 24]	Hora de inicio del viaje (decimal).
cost_transit	float, ≥ 0	Coste del billete de transporte público.
cost_driving_total	float, ≥ 0	Coste total estimado del viaje en coche.
purpose	cat. (one-hot)	Motivo del viaje: B, HBE, HBO, HBW, NHBO.
fueltype	cat. (one-hot)	Tipo de combustible: Average, Diesel, Hybrid, Petrol.

Tabla 5.4: Resultados en entrenamiento con validación cruzada de 5-folds agrupada por hogar (dataset LPMC).

Modelo	Accuracy (%)	GMPCA (%)
Random Forest	74,87	51,18
XGBoost	75,48	52,54
DNN	75,21	51,22

Tabla 5.5: Resultados en el conjunto de test independiente (dataset LPMC).

Modelo	Accuracy (%)	GMPCA (%)
Random Forest	74,10	50,33
XGBoost	74,41	51,63
DNN	74,27	50,37

6. Conclusiones y trabajo futuro

6.1. Conclusiones técnicas

6.2. Cumplimiento de objetivos

6.3. Trabajo futuro

A. Anexos

A.1. Manual de despliegue y ejecución

A.1.1. Requisitos del entorno

Versiones recomendadas

A.1.2. Arranque de servicios

OSRM, OTP, backend y frontend

A.2. Endpoints y contratos API

A.2.1. Contratos del backend

Ejemplos de petición y respuesta

A.3. Tablas de hiperparámetros y resultados completos

A.3.1. Configuraciones experimentales

Resultados detallados por experimento

A.4. Evidencias de sprints

A.4.1. Registro de tareas y decisiones

Referencias a commits y artefactos